

Bachelor Project



**Czech
Technical
University
in Prague**

F3

**Faculty of Electrical Engineering
Department of Control Engineering**

Meta-prompts for LLM Prompt Optimization

Vojtěch Klouda

**Supervisor: Ing. Jan Drchal PhD.
Field of study: Cybernetics and Robotics
May 2025**

Acknowledgements

I would like to thank my supervisor Jan Drchal for his time and helpful notes.

Additionally, I would like to thank the RCI group for access to their computation cluster.

Declaration

I declare that I have completed the submitted work independently and that I have cited all sources used.

Artificial intelligence tools were used to a limited extent during the work, for example for searching relevant literature, suggesting stylistic edits, or as a coding assistant.

In Prague, 10. May 2025

Abstract

Prompt design for Large Language Models (LLMs) remains a key bottleneck in leveraging their full capabilities, posing challenges for both expert practitioners and everyday users. In this bachelor's thesis, we develop a simple population-based hill-climber prompt optimization method built atop a custom framework for structured generation inference. Specifically, we apply prompt engineering principles to create several *meta-prompting* approaches and evaluate them across multiple tasks against a strong Instruction Induction baseline. Additionally, we survey current literature in the rapidly evolving field of prompt optimization and frame it in the broader context of training-time and inference-time scaling. Our simpler *meta-prompting* approaches perform best in experiments, outperforming both the baseline and more complex variants. Our work showcases the applicability of prompt optimization to both reasoning-intensive and creative tasks, while highlighting the sensitivity to *meta-prompt* design in LLM-based optimization. Code is available at https://github.com/skipyas0/prompt_optimizer.

Keywords: large language model, prompt optimization, inference methods, prompting techniques, structured generation

Supervisor: Ing. Jan Drchal PhD.
Resslova 307/9 Praha, E-322

Abstrakt

Tvorba promptů pro velké jazykové modely (LLM) představuje významnou překážku v jejich plném využití a je obtížná jak pro odborníky, tak pro běžné uživatele. V této bakalářské práci navrhujeme jednoduchou metodu optimalizace promptů na základě populační varianty hill-climber algoritmu, postavenou na vlastním frameworku pro strukturovanou generaci. Využíváme principy promptového inženýrství k vytvoření několika *meta-prompting* technik, které testujeme na různorodých úlohách oproti silnému referenčnímu řešení, vytvořenému pomocí Instruction Induction. Zároveň přinášíme přehled aktuální literatury v rychle se vyvíjejícím oboru optimalizace promptů a zasazujeme tuto úlohu do širšího kontextu training-time a inference-time scalingu. Naše jednodušší *meta-prompting* techniky dosahují v experimentech nejlepších výsledků a překonávají výchozí řešení i komplikovanější varianty. Naše práce ukazuje možnost využití optimalizace promptů jak v analytických, tak i v kreativních kontextech, ale zdůrazňuje citlivost optimalizace s LLM na návrh *meta-promptů*. Kód je dostupný na https://github.com/skipyas0/prompt_optimizer.

Klíčová slova: velký jazykový model, optimalizace promptů, metody inference, promptovací techniky, strukturovaná generace

Překlad názvu: Meta-prompty pro optimalizaci promptu velkého jazykového modelu

Contents

Project Specification	1	2.4.1 Exemplar optimization	28
1 Introduction	4	2.4.2 Instruction optimization	28
1.1 Background	5	2.4.3 Multi-stage and reference-free methods	35
1.2 Approach	6	3 Implementation	37
1.3 Contributions	7	3.1 Problem Formulation	38
2 Literature	8	3.2 Inference framework	41
2.1 Basics of Large Language Models	9	3.2.1 Structured generation	41
2.2 Inference-time scaling	10	3.2.2 Predict method	42
2.2.1 Chained meta-generation	11	3.2.3 Inference techniques implementation	44
2.2.2 Parallel meta-generation	14	3.3 Datasets	44
2.2.3 Step-level meta-generation	16	3.3.1 Livebench	45
2.2.4 Refinement meta-generation	17	3.3.2 Code Contests	46
2.3 Prompt engineering	19	3.3.3 Sequences	46
2.3.1 Components of a prompt	20	3.4 Evaluation Metrics	46
2.3.2 <i>Meta-prompting</i>	23	3.4.1 Metrics for Supervised Optimization	47
2.4 Prompt optimization	25	3.4.2 Metrics for Self-Supervised Optimization	47

3.5 Optimization Framework	47
3.5.1 Expansion Operator Design .	49
3.5.2 Selection Operator	55
4 Experiments	56
4.1 Supervised Optimization Operator Evaluation	57
4.1.1 Experiment setup	57
4.1.2 Experiment results	58
4.1.3 Discussion	60
4.2 Self-Supervised Optimization for Open-Ended Tasks	64
4.2.1 Limericks	64
4.2.2 Images	67
5 Conclusion	69
5.1 Conclusion	70
5.2 Future work	71
A Bibliography	73

Figures

1.1 Comparison of two prompts for a word sorting task, showcasing the goal of prompt optimization.	5	2.11 Prompt+Feedback+Score Meta-prompt	32
1.2 Diagram of our prompt optimization method.	7	2.12 Mutation <i>meta-prompt</i> , similar to paraphrasing.	34
2.1 Single-stage LLM generation: the input is processed in a single forward pass.	11	2.13 Crossover/EDA <i>meta-prompt</i> ..	34
2.2 Two-stage chained meta-generation.	12	3.1 Simple Signature	42
2.3 Parallel generation: multiple outputs are generated simultaneously.	14	3.2 Complex Signature.	42
2.4 Step-wise generation with heuristic control and memory.	16	3.3 Predict method developer prompt with highlighted prompt sections: directive, context, examples and format specifications.	43
2.5 Refinement-based generation....	18	3.4 Lamarckian Signature. Formatting specifications trimmed.	50
2.6 Comparison between pre-train+fine-tune and train+prompt paradigms.	19	3.5 Iterative Signature. Formatting specifications trimmed.	51
2.7 Directive with a placeholder....	20	3.6 Reflective Signature. Formatting specifications trimmed.	52
2.8 Using context to personalize output	21	3.7 Feedback Signature. Formatting specifications trimmed.	53
2.9 Instruction Induction.....	29	3.8 Example of a comparison of two prompts for CodeContests	54
2.10 Prompt+Score Meta-prompt ..	30	3.9 Paraphrase Signature. Formatting specifications trimmed.	54
		3.10 Formatting specifications, which are identical for all operators	55

4.1 Optimization progression for the CodeContests dataset.	60
4.2 Optimization progression for the Connections dataset.	61
4.3 Optimization progression for the Sequences dataset.	63
4.4 Average improvement of operators relative to the soft baseline.	64
4.5 Total optimization token cost per operator. Logarithmic scale.....	65
4.6 Limerick task example.....	66
4.7 Limerick optimization result ...	66
4.8 Best images from each optimization step for the words “wistful”, “hopeful” and “lonely”. .	67
4.9 Best images from each optimization step for the words “graceful”, “furious” and “vulnerable”.	68

Tables

2.1 Comparison of Zero-shot, One-shot, and Few-shot Prompting.....	22
2.2 Vastly different performance of CoT prefixes on GSM8K as per Yang et al.[1]	25
2.3 Comparison of Soft vs. Discrete Prompt Optimization. Arrow up ↑ means “better”, arrow down ↓ means “worse”.....	26
2.4 Survey of Discrete PO Methods.	27
4.1 Results for CodeContests. Listed values are calculated using equations 4.1 and 4.2 according to the metric function in equation 3.13.	59
4.2 Results for Connections. Listed values are calculated using equations 4.1 and 4.2 according to the metric function in equation 3.12.	59
4.3 Results for Sequences. Listed values are calculated using equations 4.1 and 4.2 according to the metric function in equation 3.14.	62

DECLARATION

I, the undersigned

Student's surname, given name(s): Klouda Vojtěch
Personal number: 507654
Programme name: Cybernetics and Robotics

declare that I have elaborated the bachelor's thesis entitled

Meta-prompts for LLM Prompt Optimizatio

independently, and have cited all information sources used in accordance with the Methodological Instruction on the Observance of Ethical Principles in the Preparation of University Theses and with the Framework Rules for the Use of Artificial Intelligence at CTU for Academic and Pedagogical Purposes in Bachelor's and Continuing Master's Programmes.

I declare that I used artificial intelligence tools during the preparation and writing of this thesis. I verified the generated content. I hereby confirm that I am aware of the fact that I am fully responsible for the contents of the thesis.

In Prague on 22.05.2025

Vojtěch Klouda

.....
student's signature

I. Personal and study details

Student's name: **Klouda Vojtěch** Personal ID number: **507654**
Faculty / Institute: **Faculty of Electrical Engineering**
Department / Institute: **Department of Control Engineering**
Study program: **Cybernetics and Robotics**

II. Bachelor's thesis details

Bachelor's thesis title in English:

Meta-prompts for LLM Prompt Optimization

Bachelor's thesis title in Czech:

Meta-prompty pro optimalizaci promptu velkého jazykového modelu

Name and workplace of bachelor's thesis supervisor:

Ing. Jan Drchal, Ph.D. Artificial Intelligence Center FEE

Name and workplace of second bachelor's thesis supervisor or consultant:

Date of bachelor's thesis assignment: **24.01.2025** Deadline for bachelor thesis submission: **23.05.2025**

Assignment valid until: **do konce letního semestru 2025/2026**

doc. Ing. Zdeněk Hurák, Ph.D.
Head of department's signature

prof. Mgr. Petr Páta, Ph.D.
Vice-dean's signature on behalf of the Dean

III. Assignment receipt

The student acknowledges that the bachelor's thesis is an individual work.
The student must produce his thesis without the assistance of others, with the exception of provided consultations.
Within the bachelor's thesis, the author must state the names of consultants and include a list of references.

Date of assignment receipt

Student's signature

I. Personal and study details

Student's name: **Klouda Vojtěch** Personal ID number: **507654**
Faculty / Institute: **Faculty of Electrical Engineering**
Department / Institute: **Department of Control Engineering**
Study program: **Cybernetics and Robotics**

II. Bachelor's thesis details

Bachelor's thesis title in English:

Meta-prompts for LLM Prompt Optimization

Bachelor's thesis title in Czech:

Meta-prompty pro optimalizaci promptu velkého jazykového modelu

Guidelines:

This thesis explores the design of so-called meta-prompts, which are important modules of prompt optimization methods for LLMs. Focus on different approaches to 1) generate initial solutions (prompts) and 2) improve existing solutions in a supervised learning setting of prompt optimization (given a small training dataset and additional test data.)

- 1) Review current methods for prompt optimization and meta-prompt design.
- 2) Pay special attention to black-box optimization techniques and prompting approaches such as Chain of Thought, Tree of Thought, ReAct, or Reflexion.
- 3) Design and implement several meta-prompting approaches to generate initial and improve existing prompts.
- 3) Implement a basic black-box optimization method (e.g., a hill-climber) to the meta-prompting operators.
- 4) Identify existing datasets or create new ones, focusing on tasks with complex input-output transformation relationships.
- 5) Design an evaluation methodology and conduct experiments to assess the performance of the proposed method.

Bibliography / sources:

- [1] Guo, Qingyan, et al. "Connecting large language models with evolutionary algorithms yields powerful prompt optimizers." arXiv preprint arXiv:2309.08532 (2023). [2] Cui, Wendi, et al. "PhaseEvo: Towards Unified Long-Context Prompt Optimization for Large Language Models." First Workshop on Long-Context Foundation Models@ ICML 2024.
- [2] de Wynter, Adrian, et al. "On Meta-Prompting." arXiv preprint arXiv:2312.06562 (2023).
- [3] Hou, Yutai, et al. "MetaPrompting: Learning to learn better prompts." arXiv preprint arXiv:2209.11486 (2022).
- [4] Pryzant, Reid, et al. "Automatic Prompt Optimization with "Gradient Descent" and Beam Search." Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 2023.
- [5] Soylu, Dilara, Christopher Potts, and Omar Khattab. "Fine-Tuning and Prompt Optimization: Two Great Steps that Work Better Together." Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024.



Chapter 1

Introduction

1.1 Background

In recent years, Large Language Models (LLMs) have permeated the Natural Language Processing research landscape as well as into the general public. Already achieving human-like performance at a wide variety of tasks[2], they are bound by scaling laws[3] which predict performance gained with increased computation expenditure. This is currently fueling massive investments into computation capacity by industry players.

With costs of training new state-of-the-art foundational LLMs rising rapidly, research has turned to inference-time scaling[4], based on post-training[5][6] utilizing reinforcement learning and supervised fine-tuning, and prompting techniques[7].

LLMs are notoriously sensitive to prompt inputs[8][9] and manually creating performant prompts is very difficult[10] for expert users and members of the general public. These problems inspired research into how prompt design could be automated, either by generating prompts from task examples[11] or by further optimizing them by establishing a LLM-powered feedback loop[1][12][13]. This forms the basis of a novel research branch dubbed compile-time scaling[14], representing a middle ground between the training-time and inference-time scaling paradigms.

Input: sewer, window, pipe, drain, duct, smile, egg, knuckles

Gold: drain, duct, pipe, sewer ; egg, knuckles, smile, window

Unoptimized prompt:

Sort the words into 2 categories by meaning.

Optimized prompt:

You are given several words that can be sorted into groups of 4 based on several characteristics, for example:
 meaning, common usage, number of syllables, rhyming etc.
 Output the words sorted into categories of 4. Separate words with commas “,” and groups with semicolons “;”.

Figure 1.1: Comparison of two prompts for a word sorting task, showcasing the goal of prompt optimization.

To better illustrate the concept of prompt optimization, in figure 1.1 we show two example prompts for a word categorization task from the Livebench[15] benchmark. In this task, the LLM has to sort 8, 12 or 16 words into groups of 4 based on non-trivial characteristics.

Notice the unoptimized prompts fails to reflect the variability in the number of input words and does not enforce the necessary formatting. Furthermore, it only instructs the LLM to sort the words by meaning, possibly overlooking other aspects. The goal of the optimization process is to alleviate all these issues and produce a more complete optimized prompt.

To summarize, optimization using LLMs[16][17] and particularly prompt optimization presents an exciting intersection between deep learning and traditional optimization algorithms, like evolutionary algorithms[18][19][20] and other metaheuristics[21].

1.2 Approach

The main goal of this bachelor’s thesis is to gain hands-on experience with automatic prompt optimization by designing a modular optimizer framework. With a simple underlying architecture, a population-based hill-climber algorithm, we focus our attention on the design of *meta-prompts*, or prompts that generate other prompts. Using these, we design a lineup of different population expansion operators.

In figure 1.2 we show our optimization architecture, including the different expansion operators and initialization using instruction induction from task samples[11]. In each optimization step, the population is pruned by removing duplicates. These are found using relative edit distance. Evaluation is done using task-specific gold label-based metrics or in a self-supervised setting using pairwise comparisons. Optimization loop continues for a specified number of iterations.

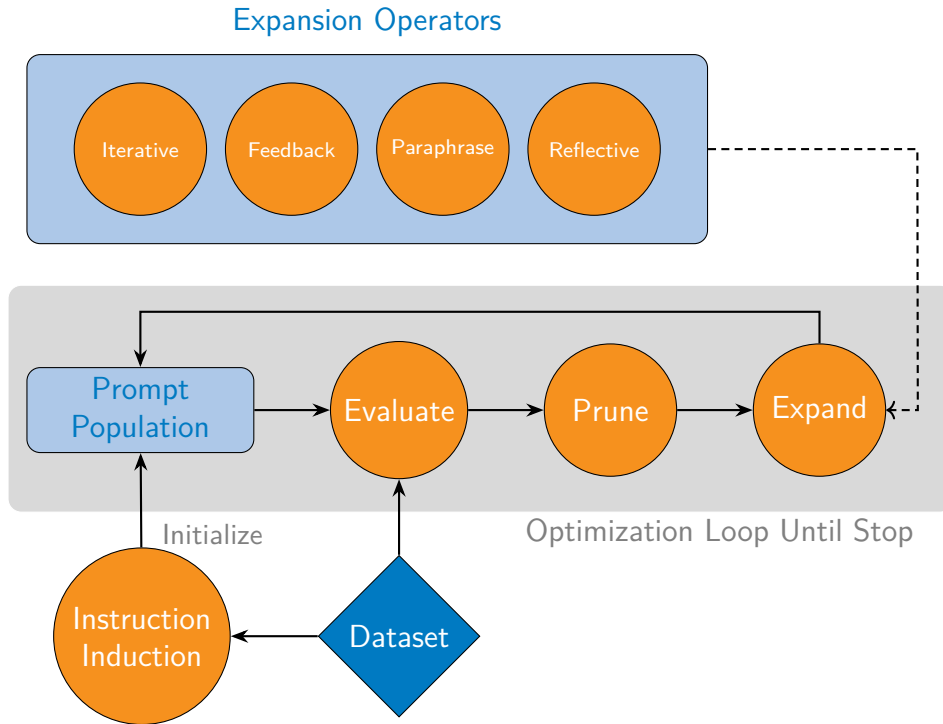


Figure 1.2: Diagram of our prompt optimization method.

1.3 Contributions

In this section, we will briefly list the main contributions of this bachelor's thesis, which include the following.

1. **Literature survey:** We provide an extensive overview of current literature on prompt optimization and frame it in the broader context of inference-time scaling algorithms.
2. **Structured generation:** We explicitly design our optimization framework around a structured generation framework that also implements several inference-time techniques.
3. **Comprehensive evaluation:** All 4 variants of our method are evaluated on 3 diverse tasks, including a custom numerical pattern recognition task. Evaluation is conducted using both gold label-based metrics and in a self-supervised setting based on pairwise comparisons. The latter is also applied to creative tasks, including image generation.



Chapter 2

Literature

2.1 Basics of Large Language Models

In Natural Language Processing (NLP), language models are machine learning models that model statistical dependencies in language. Specifically Large Language Models (LLMs) are language models with many layers and massive parameter counts, often in the billions, trained on enormous amounts of text.

Current state-of-the-art models use the Transformer[22] architecture and its derivatives. In its essence, this architecture chains attention mechanisms with fully connected layers. This simple architecture, when applied on massive scale using vast computation and data resources, is at the root of the current LLM revolution in the fields of NLP and artificial intelligence.

LLMs predict the next token in a sequence of tokens. Tokens can represent a letter or a word, but current models utilize sub-word tokens, similar in length to syllables. The meaning of these tokens can be inferred from relative position to other tokens in training text and encoded into vector form using an algorithm like Word2Vec[23].

When we give a sequence of tokens as an input to an LLM, it produces a probability distribution of possible next tokens. This happens both during training, where the probability distribution is used to update the model's weights to minimize training loss, and during *inference*. Inference is the process of using a frozen LLM to generate text.

During inference, a sampling algorithm is employed to pick the next token from the probability distribution. Usually an interpolation between greedy and uniform sampling is employed. This process is called temperature sampling and introduces an important hyperparameter - *temperature* τ . Temperature influences the randomness of next token prediction and affects the outputs' qualities, like creativity and novelty. For $\tau = 0$, the sampling process is deterministic in theory, but in practice variance remains due to numerical errors.

The input sequence is generally called a *prompt*. The model recursively generates next tokens, forming an output sequence, which is also referred to as a *completion*. Generation continues until a special **stop** token is generated or until a token limit is reached.

In LLM training, loss has been empirically found to scale with computation

resources used[3]. This is referred to as training-time scaling. With diminishing returns, research has turned to inference-time scaling, characterized by expending more resources during inference, for example by post-training the LLM to begin completions with a reasoning chain. This brought impressive results particularly in reasoning-heavy domains. The post-training is done using both reinforcement learning and supervised finetuning.

2.2 Inference-time scaling

Inference-time scaling or test-time scaling is a paradigm that has gained traction in recent years with the advent of dedicated reasoning models[5][6]. As opposed to training-time scaling, where the performance of models scales with training times, model parameter counts and dataset sizes[3], inference-time scaling aims to improve performance by dedicating more resources to each inference call.

At their heart LLMs are probabilistic models over sequences and to generate a sequence they employ generation algorithms. Welleck et al.[4] provide an overview of these generation algorithms and then frame more advanced inference-time techniques as meta-generations, or strategies that employ sub-generators.

Most generation algorithms attempt to find either highly probable sequences (MAP algorithms) or sample from the model's distribution. The simplest MAP algorithm is greedy decoding, which recursively finds the next token with the highest probability in the distribution. An example of algorithms that sample from the model's distribution is the ancestral sampling algorithm[4].

A generalization of greedy decoding is the beam search algorithm which maintains a data structure of possible prefixes and in each step expands them and scores them. An example[24] of a beam search algorithm can identify decoding branches where the model employs a reasoning chain to solve a given task. Authors of this algorithm found that answer tokens found in the decoding paths with a reasoning chain have greater token probabilities. This means that the model shows greater confidence in its answer having reasoned about it beforehand. In general, beam search improves on simple greedy decoding but at a high computational cost[4].

Another class of generation algorithms are those which interpolate between multiple categories of sampling algorithms. Temperature sampling,

which outperforms other adapters in input-output tasks like code generation and translation, is an interpolation between greedy sampling and uniform sampling.

Interpolating between ancestral sampling and simple greedy sampling gave rise to decoding algorithms such as nucleus, top-k and η - and ϵ -sampling[4].

When we require a structured output, for example a JSON data structure following a JSON schema, we can utilize parser-based decoding, which enforces a structural requirement. This can however come at worsened performance when using inflexible templates[25].

These low-level generators can be interconnected into more complex techniques, which Welleck et al. dub meta-generators[4]. We will maintain their terminology and discuss different sequence-level meta-generation algorithms. We will omit further discussion of token-level methods as they are irrelevant to the main topic of this thesis.

These strategies can be divided into the categories of chained, parallel, step-level, and refinement-based meta-generators.

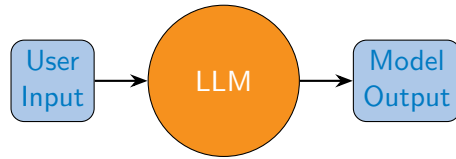


Figure 2.1: Single-stage LLM generation: the input is processed in a single forward pass.

2.2.1 Chained meta-generation

Chained meta-generation is the composition of several sub-generators in sequence. These can be LLM calls or other functions that use previous completions, such as a code execution function[26] or a tool for interaction with an arbitrary environment or a data source[27]. The sub-generators can be implemented as several LLM calls or with a single call given sufficient instructions in the prompt[28].

In figure 2.2, see a diagram of chained meta-generation. In its simplest form, there would be no environment interaction and the LLM would just

split processing the input into planning (thinking) and finishing stages. An example of such method is Chain-of-thought[29][30].

Alternatively, the partial output can be used to interact with a non-LLM environment, like a code execution tool or a searchable data corpus. This augments the final output synthesis, forming techniques like Program of Thoughts[26] and ReAct[27]. Furthermore, we can add a history of past actions and environment observations to the architecture in 2.2 with a loop-back, forming a basic agentic loop.

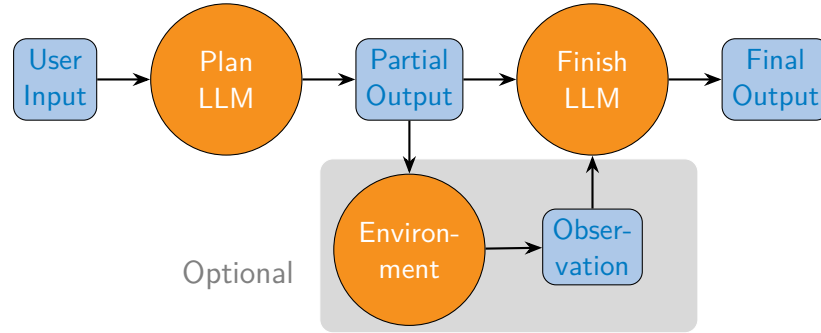


Figure 2.2: Two-stage chained meta-generation.

In its essence, the model is a left-to-right text completion engine. We can make the analogy with human thinking modes, where it is said that humans have a fast automatic "System 1" mode and a slow and deliberate "System 2" mode[31]. In direct-QA mode, the LLM can underestimate the difficulty of the task[24] and stay in the "System 1" thinking mode. Simple greedy decoding paths mostly do not contain a reasoning chain[24], which means the model tends to make a guess, staying in "System 1". By crafting a good prompt that instructs the model to reason we can shift the model from "System 1" to "System 2" thinking.

Another reason for the effectiveness of chained generation is that in LLM training some concepts and variables are observed more frequently than others[32]. This discrepancy hurts performance in direct-QA scenarios where the relevant variables are rarely seen together in training. With CoT, models can incrementally chain known dependencies and bridge conceptional gaps.

■ Chain-of-thought

Chain-of-thought[30] (CoT) is an LLM prompting technique that works by inducing a coherent series of intermediate reasoning steps that lead to the final answer for a problem, thereby increasing computation time. Upon its

introduction, it brought a dramatic performance increase on arithmetic tasks, where models previously struggled. This enhanced capability comes at the cost of longer and more computationally expensive outputs[33]. Improvements are more noticeable for more complicated problems[30].

CoT can be elicited by prompting techniques - few-shot with reasoning step demonstrations or zero-shot with specific instructions[24]. First CoT methods[30] involved one/few-shot prompting. Although effective, this requires human engineering of multistep reasoning prompts. This method is also highly sensitive to prompt design with performance deteriorating for mismatched prompt example and task question types[29]. For this method, authors found that CoT is an emergent capability of model scale and did not observe benefits for small models[30].

On the other hand, zero-shot prompting induces a reasoning chain with a simple prompt like "Let's think step-by-step", making it versatile and task-agnostic[29]. Similar prompts also improve reasoning performance and some research[1] has been done on finding the optimal CoT prompt.

Apart from prompting, CoT can be elicited and improved by model training or tuning. This method, requiring a significant amount of reasoning data[24], has gained traction with the development of dedicated reasoning models like OpenAI's o1[5] or Deepseek-R1[6].

Using methods such as supervised fine-tuning (SFT) or reinforcement learning (RL), the model is trained to automatically produce longer reasoning chains, often bound in dedicated "thought" tags or tokens. These models have shown significant performance boosts on reasoning benchmarks[5][6].

Models similar to o1 all primarily extend solution length by self-revision[34]. After finishing a thought process, the model tries to self-revise, which is marked by words such as "Wait" or "Alternatively". The model then tries to spot mistakes or inconsistencies in its reasoning or propose an alternative solution. Self-revision ability is thus a key factor in the effectiveness of sequential scaling for reasoning models[34].

Longer reasoning chains mean more computing power spent at inference. How far can we take this sequential scaling? In their study, Zeng et al.[34] argue that longer CoTs do not consistently improve accuracy of reasoning models. Furthermore, they find that the average length of correct solutions is shorter than that of incorrect ones.

Because self-revision accounts for most of the CoT length, the effectiveness of the method relies on the model’s ability to self-revise. Authors of this paper argue that the self-revision ability of models is insufficient because they demonstrate limited capacity to correct their answers on their own, without external environment feedback. Some models on some tasks are even more likely to change a correct answer to an incorrect one than vice versa.

Further research by Liu et al.[35] suggests that for some tasks CoT can be detrimental. Their experiments proved their hypothesis that CoT hurts performance on tasks where humans do better without deliberation and where the nature of LLM, like the much greater context memory, does not provide an advantage over human thinking. This phenomenon was observed on tasks like facial recognition, implicit statistical learning or pattern recognition.

2.2.2 Parallel meta-generation

Parallel meta-generation involves multiple generations concurrently. The final answer can then be chosen by voting or a reward model, or constructed from the ensemble of generations[4].

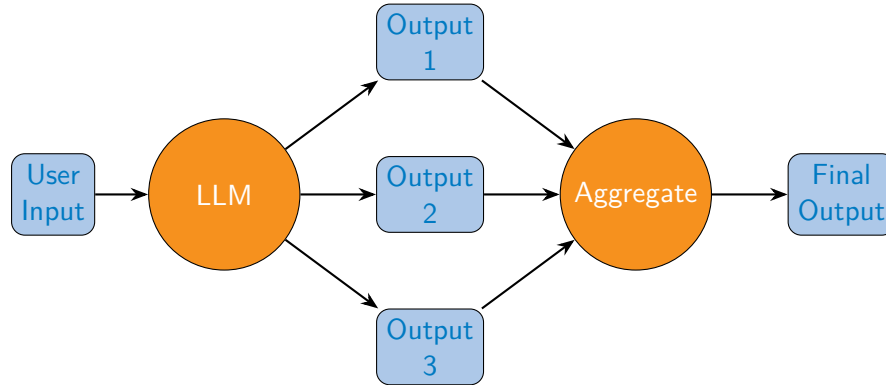


Figure 2.3: Parallel generation: multiple outputs are generated simultaneously.

Parallel meta-generation allows weaker models to outperform bigger and more expensive models[33]. This can sometimes reduce cost as multiple samples with a smaller model are cheaper than a single sample with a more capable model. This is helped by the fact that parallel sampling can make use of batching and other system throughput optimizations available for parallel inference[33].

One of such techniques is self-consistency[36] (SC), a simple method which builds upon CoT to aggregate answers from diverse reasoning chains and

selects the best one based on majority voting. It significantly improves accuracy in a range of arithmetic and commonsense reasoning tasks at the cost of increased computation expenditure[36]. The effectiveness of SC with majority-voting comes from the fact that, for tasks with objective answers, there are often more ways to be wrong than to be right.

For our next discussion of SC and related methods we will formally define and compare the terms *coverage* $C_{\mathcal{D}}$ and *accuracy* $A_{\mathcal{D}}$ for a dataset $\mathcal{D}$. Given a language model $\mathcal{M}$, a task query $q_k \in \mathcal{D}$ and an instructional prompt P , we can define the generation collection of length n as

$$Y_k = \{y_{jk} \mid j \in 1, \dots, n\}, \quad (2.1)$$

$$y_{jk} \sim \mathcal{M}(P \mid q_k). \quad (2.2)$$

For objective tasks, we can check the correctness with a metric $\mathcal{G}$

$$\mathcal{G}_k(y_{jk}, q_k) = \begin{cases} 1.0 & y_{jk} \text{ is the correct answer for } q_k \\ 0.0 & y_{jk} \text{ is an incorrect answer for } q_k. \end{cases} \quad (2.3)$$

To choose the final answer, we will define an answer selection function $\mathcal{S}(Y)$. This can be a majority vote selection function or some reward-based method.

We can now define *coverage* $C_{\mathcal{D}}$ and *accuracy* $A_{\mathcal{D}}$ as

$$C_{\mathcal{D}} = \frac{1}{|\mathcal{D}|} \sum_{q_k \in \mathcal{D}} \max_{j=1, \dots, n} \mathcal{G}_k(y_{jk}, q_k) \quad (2.4)$$

$$A_{\mathcal{D}} = \frac{1}{|\mathcal{D}|} \sum_{q_k \in \mathcal{D}} \mathcal{G}_k(\mathcal{S}(Y_k), q_k). \quad (2.5)$$

In plain language, coverage is the fraction of the tasks where at least one sample results in a correct answer, whereas accuracy is the fraction of the tasks where a correct answer is selected by the algorithm as the final answer.

It is easy to see why coverage might rise as we increase the amount of samples n in SC generation. One could imagine that as letting students answer with their top n guesses for each question on a test. Indeed, research[33] has found that the relationship of coverage and the number of samples can be modeled by an exponentiated power law, suggesting a scaling law for inference similar to the training scaling laws[3].

However, coverage alone is not enough to paint the complete picture. What good is it to have a large collection which contains a correct answer if we cannot verify which one is correct. Parallel scaling with large sample collections is only useful if the correct samples in a collection can be identified[33][34].

The accuracy gain of SC tends to saturate quickly as we increase the number of paths[36]. Although coverage rises, it diverges[33] from accuracy as the algorithm is unable to select the correct answer from the collection. This highlights the necessity to develop better answer selection mechanisms than simple majority voting and automatic answer verification methods.

Zeng et al.[34] make use of the fact that correct solutions have shorter CoT on average and develop a length-weighted majority vote that outperforms simple majority voting on challenging math benchmarks. GLaPE[37] is a method for gold label-agnostic evaluation which makes use of the fact that incorrect answers tend to have higher variability, but fails in scenarios where an incorrect answer is produced consistently.

2.2.3 Step-level meta-generation

Maintaining the terminology of Welleck et al.[4], step-level meta-generation algorithms implement search on the generation state-space. This can be done on the token level or on the level of longer sequences, but in this section we will focus on the latter.

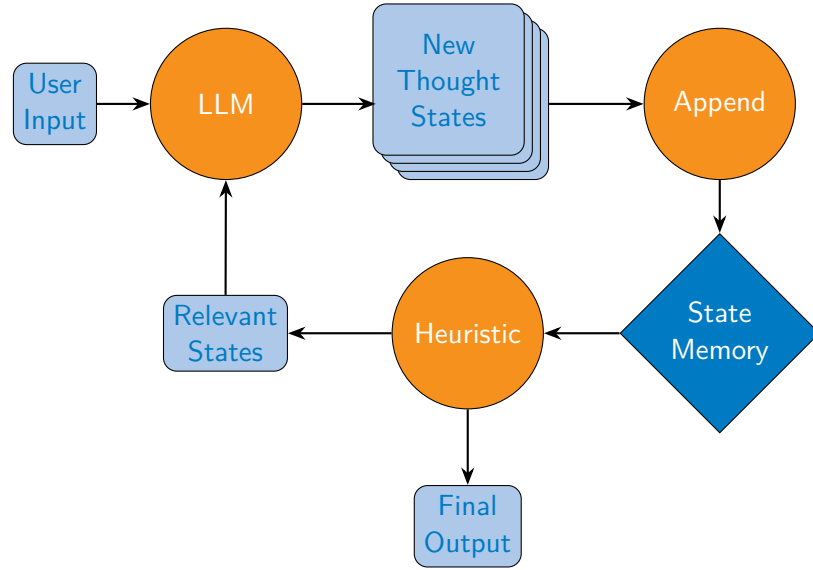


Figure 2.4: Step-wise generation with heuristic control and memory.

Previously discussed inference-time scaling techniques all rely on sequential or parallel linear thought processes. They do not explore different continuations within a thought process and do not make use of planning, look-ahead,

or backtracking[31]. These methods also do not allow combining the flow of reasoning upon discovering new insights, something humans utilize when solving problems[38].

By generalizing CoT[30][29] into a tree structure, Yao et al.[31] present Tree of Thoughts (ToT), a technique which maintains a tree of states, representing individual thoughts. In this tree, each node is a thought in a form of a coherent language sequence, serving as an intermediate step in the reasoning process. For traversing the tree, a general tree-search algorithm, such as breadth-first or depth-first search, can be employed.

An important parameter in ToT is the branching factor. Unlike the standard tasks typically tackled by tree search algorithms where the number of possible actions at each node is finite, each call to LLM can yield a new output even for the same input, making each node’s branching factor theoretically infinite[39]. Misaki et al.[39] argue that fixed-width multi-turn methods exhibit diminishing gains and develop a tree search method with an adaptive branching factor, leading to a more balanced exploration and exploitation capability.

Although ToT allows for planning and backtracking from unpromising thought chains, its structure is still too rigid[38]. For example, it is not possible to combine thoughts from independent branches of the tree. Graph of Thoughts[38] (GoT) is a framework that models the reasoning process as a heterogeneous directed graph where each vertex is a thought containing a (partial) solution and edges are dependencies between these thoughts[38].

In both chain- and tree-based inference-time scaling methods, a substantial amount of compute power is allocated to processing historical information that are not beneficial to the reasoning process. To alleviate this, Atom of Thoughts[40] (AoT) iteratively decomposes the current question into a directed acyclic graph. The graph consists of sub-questions which, depending on whether they have dependencies, are dependent or independent. All the independent questions can be answered directly, and their answers added to the question context. The remaining sub-questions are then contracted into a new current question.

2.2.4 Refinement meta-generation

The last category of meta-generation algorithms are refinement algorithms. Refinement algorithms work by alternating between generation and refinement.

The refiner generates a revised version of the output based on past versions and additional information such as intrinsic or extrinsic feedback or environment observations[4]. Intrinsic refinement comes from the model inspecting its own answers, as seen in 2.5. Optionally, output candidates and feedback can be added into a memory component for later reference.

As we discussed in 2.2.1, models struggle with self-revision and rarely modify their answers in long reasoning chains. Feedback from general models is ineffective compared to dedicated feedback models or other quality feedback sources[41].

For extrinsic refinement, the model can utilize external information which can lead to a potential gain with refinement[4]. When the **Evaluator** LLM component also includes environment interaction, figure 2.5 can be seen to depict extrinsic refinement.

One example of a refinement-based framework, Reflexion[42], converts binary or scalar feedback from the environment into verbal feedback in the form of a textual summary. This feedback is then added as additional context for the LLM agent, e.g. CoT or ReAct module, in the next episode. Reflexion improves performance over strong baselines on sequential decision-making, reasoning and programming tasks[42].

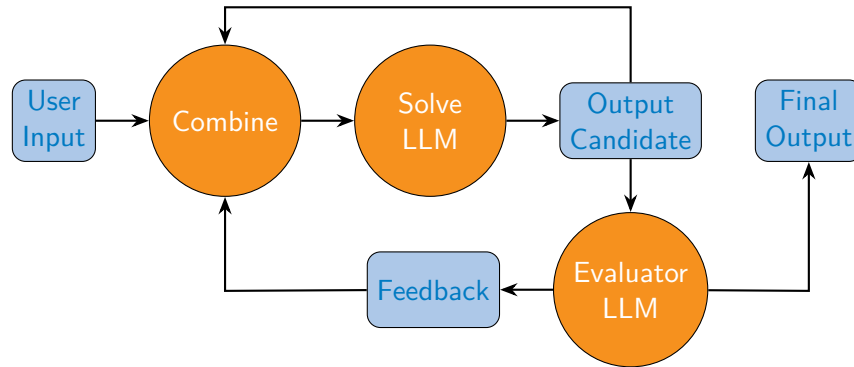


Figure 2.5: Refinement-based generation.

2.3 Prompt engineering

By prompt engineering we mean crafting a suitable instruction which, when combined with a query, incites a LLM response that matches our requirements. Our task requirements can for example be: a) obtaining the correct answer for a mathematical problem, b) fixing a bug in a code base, or c) explaining the contents of an image. Each of these tasks needs a separate instructional prompt P which can then be used with multiple queries, representing specific task instances.

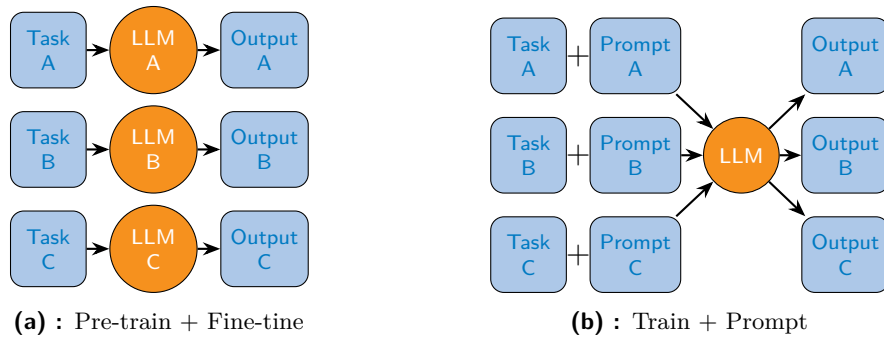


Figure 2.6: Comparison between pre-train+fine-tune and train+prompt paradigms.

This signifies a shift from the training and fine-tuning paradigm, where a base model is first trained on a large corpus of data and then adapted for a specific task with supervised fine-tuning. This process requires a substantial amount of training data and computation power, making specialized LLMs unsuitable for many users and for applications, where extensive data collection is infeasible.

We illustrate this shift in figure 2.6. Although prompt design is difficult, it is much less resource-intensive than fine-tuning.

Since the inception of modern LLMs, prompt engineering has evolved into a field of its own. Current LLM systems, often containing multiple chained and interlinked models, require robust and well-thought-out prompts at each step. Indeed, in many modern LLM applications, prompts have become programs themselves[14].

In this section, we will briefly cover the most notable prompt engineering techniques, which we will then be able to utilize in our study of automatic prompt optimization.

2.3.1 Components of a prompt

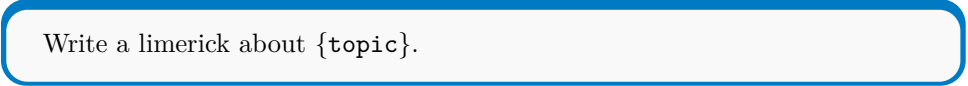
We can roughly dissect a prompt into 4 different components[7].

- **Directive:** The main task of the prompt, e.g., *"Write an email to a coworker."*
- **Context:** Everything necessary or beneficial to completing the directive, e.g., *"I was supposed to send a report to my boss, but I forgot."*
- **Examples:** How you would have solved a similar task, e.g. a past email on a similar topic.
- **Output specifications:** Style and format instructions, e.g., *"Respond with three paragraphs in formal style with tasteful emojis."*

Although flexible and sometimes blended together, prompts often follow this structure and order. In more technical applications where prompts might become more convoluted, it might be beneficial to use tags or delimiters to explicitly separate components. Models often reward prompts with a more code-like structure[10]. We now discuss each component in detail.

Directive

The directive should be a clear and objective description of the task, assuming that the model already knows how to solve it[43]. Specific requirements that narrow the scope should be avoided and left for other components.



Write a limerick about {topic}.

Figure 2.7: Directive with a placeholder

In cases where the prompt serves as a prompt template, meaning it can be reused with various data points, the directive can include a placeholder. For example consider the directive in figure 2.7, which could be reused for multiple values of `topic`.

Context

Context should provide all the background information relevant to the task at hand. The user can include more information about barriers which prevent them from solving the problem on their own, define the target audience or attach relevant documents.

In figure 2.8, we show a prompt which asks the model to summarize an article, with the context component providing further specifications.

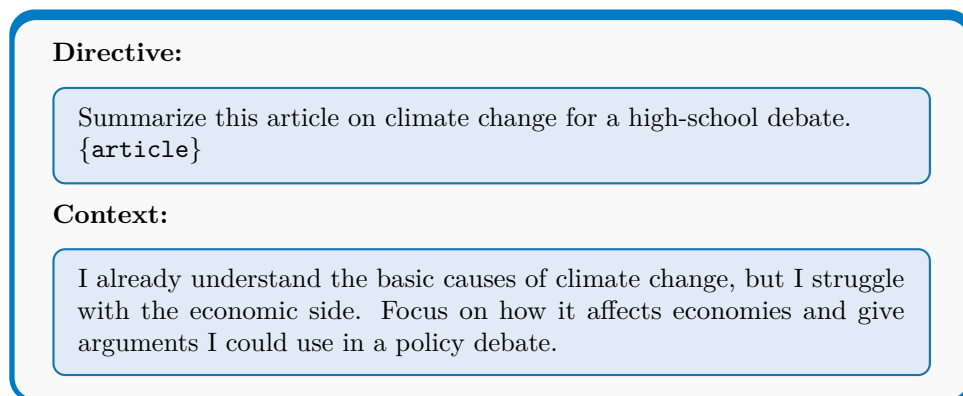


Figure 2.8: Using context to personalize output

In this example, the initial instruction serves as the directive, while the second component provides contextual information about the user's prior knowledge and objectives. Without this context, the model may produce a generic summary, overlooking the user's interest in economic impacts and failing to discuss arguments relevant for a policy-oriented debate.

The context can become the endpoint for retrieval pipelines, which search a data source for relevant documents, or memory mechanisms, which gather personal information about the user from other conversations.

Another possible feature often used in the context component is a memetic proxy[43], like role-assignment. Instead of writing a long instruction covering all the requirements and assumptions behind someone being an "experienced business analyst", we can just say "You are an experienced business analyst". In this way, we can instruct the model to adopt an identity or an expertise level. This primes the model to consistently use more technical language in its response.

■ Examples

By providing examples of solutions to similar tasks, we can condition the LLM to generate further examples from that distribution[16], increasing the chance of a suitable completion. Furthermore, examples make the decoding more robust and decrease prompt sensitivity[8].

Prompting Type	Description
Zero-shot Prompting	Prompt has no examples. Model relies on instructions and pre-trained knowledge.
One-shot Prompting	Prompt has one example to guide the model.
Few-shot Prompting	Prompt includes a few examples.

Table 2.1: Comparison of Zero-shot, One-shot, and Few-shot Prompting

Table 2.1 shows the agreed-upon terminology for prompts with examples. The concept of adding examples to the prompt is also called In-Context Learning (ICL). In some cases, Few-shot prompting can be effective even without the use of other instructions[44]. When adding examples to a prompt, we have to pay attention to several aspects[7].

- **Exemplar quantity:** More is better with diminishing returns.
- **Exemplar ordering:** Models tend to pay more attention to the last examples.
- **Exemplar label distribution:** Unbalanced labels in examples skew model generation.
- **Exemplar label quality:** It is unclear whether incorrect examples hurt performance.
- **Exemplar format:** Optimal format may vary across tasks.
- **Exemplar similarity:** Effect of exemplar similarity depends on the situation.

Contrary to Brown et al.[44] who interpret the effectiveness of few-shot prompting as the model learning the task by observing examples, later research[43] suggests that examples merely allow the LLM to more precisely locate the task in its learned task space. Still, the authors argued for the use of examples for redundancy enforcing the desired behavior[43].

■ Output specifications

Using this component, we guide the structure, tone, style and formatting of the LLM’s answer. A common technique, discussed in 2.2.1 is inducing reasoning with a CoT prompt, like "Let’s think step-by-step", or instructing the LLM to first plan the solution and then execute it.

We can influence the length of the answer, ask the model to be formal or humorous, request specific formatting, like the use of $\text{\LaTeX}$ equations, or have it answer in a JSON format for a machine-readable response.

With innumerable options for output specification, users should test multiple configurations as changing the output format can heavily influence the final prediction accuracy[9].

■ 2.3.2 *Meta-prompting*

Before we proceed, we first need to overview the terminology discrepancies in contemporary research. *Meta-prompts* (also meta prompts or metaprompts) were first coined as a term by Reynolds and McDonell[43]. Since then, this term was used in multiple contexts.

1. **Task-agnostic zero-shot prompt:** In contrast to task-sample specific few-shot prompting, *meta-prompts* were used to mean a “task-agnostic zero-shot prompt”, or “seeds encapsulating a more general intention that will unfold into a specific prompt when combined with the task question” [43].
2. **Natural language meta-procedure:** Extending 1, Zhang et al.[45] define *meta-prompts* as example-agnostic prompts designed to capture the reasoning structure of a specific category of tasks. They do this by employing a typed and structured prompt, resembling control flow templates and often expressed in JSON-like structures. This is similar to how DSPy[28] implements LLM calls with function signatures.
3. **Soft prompt optimization method:** In a parallel branch of research, MetaPrompting[46] was used as a name of a soft token prompt optimization method. Soft token prompts are, in contrast to human-readable textual discrete token prompts, raw floating-point vector inputs to the

first layer of a language model. This makes soft token prompt optimization amenable to gradient optimization.

4. **Prompt generator:** In automatic prompt engineering and prompt optimization literature, *meta-prompt* is a prompt that generates prompts[47]. This can be seen as an application of 2 to the task of prompt generation[45].

While both 1 and 2 are of interest to us, in this thesis, we will treat *meta-prompts* as prompt generators.

■ *Meta-prompt as a prompt generator*

Prompt optimization literature[48] treats *meta-prompts* simply as a prompt which generates other prompts. As prompt generation is a complex reasoning-intensive language generation task[49], all the principles regarding prompt design for other difficult tasks apply to *meta-prompts* as well.

Meta-prompts are usually templates for other data, such as examples of the task for Instruction Induction[11], past prompt generations along with their scores[1] and critiques of prompt outputs[13] or text descriptions of the task[49]. Some research also utilizes professional task advice[48] or a prompt engineering tutorial[49]. Other methods used seed phrases, like a thinking style[20], to steer the generation.

In one of the few theoretically rigorous works on the subject, de Wynter et al.[47] formalize meta-prompts using category theory. Using this formalism, they show the possibility of creating a general purpose meta-prompt as well as suggesting that such meta-prompt will perform better than task-specific prompts in a wide range of applications.

2.4 Prompt optimization

In a previous section on inference-time algorithms, we outlined several methods for improving the performance of LLM generation and contrasted it to the traditional training-time scaling paradigm. In this section, we will discuss prompt optimization. This technique fits in neither training- nor inference-time scaling techniques, as the computation allocated on optimizing our prompts can then be amortized over multiple uses of the prompt. We will refer to this as compile-time[14] scaling.

We will first discuss the motivation for automatizing the prompt engineering process. Prompt engineering is a largely empirical field without rigorous foundations. Although research agrees on some best practices, creating effective prompts often requires substantial trial-and-error experimentation and deep task-specific knowledge[50].

As LLM use permeates into the general public, research[10] notes difficulties of the average user with prompt design and evaluation as users bring expectations from human-to-human interactions to prompt design. These include the expectation that semantically equivalent instructions should produce equivalent results. This however does not hold and even minute alterations to the prompt, like adding a single space to the end, can dramatically influence the answer[8][9]. In table 2.2 we illustrate the prompt performance variability with examples of zero-shot CoT[29] prefixes have vastly different performance on mathematical tasks.

Prompt	GSM8K accuracy (%)
“Let’s think step by step.”	71.8
“Let’s solve the problem together.”	60.5
“Let’s work together to solve this problem step by step.”	49.4

Table 2.2: Vastly different performance of CoT prefixes on GSM8K as per Yang et al.[1]

The average user is not used to the meticulous process of systematic program crafting and debugging. It seems that some coding proficiency is, in a way, a prerequisite for prompt engineering. With these limitations barring the general public from utilizing LLMs to their full potential, automatic prompt engineering and prompt optimization (PO) have become an exceptionally active research field in recent years.

We can divide the PO research into two distinct branches depending on whether they treat prompts as sequences of discrete tokens or as soft embedding vectors. Both approaches have their pros and cons, summarized in table 2.3.

Feature	Soft Optimization	Discrete Optimization
Gradient use	✓	×
Interpretability	↓	↑
Transferability	↓	↑
Cost	↓	↑
Usable with APIs	×	✓

Table 2.3: Comparison of Soft vs. Discrete Prompt Optimization. Arrow up ↑ means “better”, arrow down ↓ means “worse”.

Although soft PO is more effective as it allows for the use of continuous optimization methods such as gradient descent, with the increasing size of models it gets more expensive akin to fine-tuning. Furthermore, many of the most capable models are only accessible through proprietary APIs, which rarely allow access to the inner states of the LLM, rendering soft PO unusable. Optimizing prompts for an LLM hidden behind an API is inherently a black-box problem. In this thesis, we will study methods of discrete PO.

Table 2.4 offers an overview of discrete PO literature. Note that only selected methods are included based on 1) being somewhat comparable, and 2) being relevant to the implementation part of this thesis. Also note that, due to space-constraints, only the most notable aspects of the methods are included. For a more comprehensive overview of PO methods, we refer the reader to a survey[48] by Ramnath et al.

Method	Initialization	Selection	Expansion	Notes
APE[12]	instruction induction	filter top-k prompts	paraphrasing	iterating does not help
ProTeGi[51]	manual initial prompt	statistical resource allocation algorithms	critique of a prompt as a "gradient"	state-of-the-art according to[52]
OPRO[1]	baseline prompt, instruction induction	filter top-k prompts	<i>meta-prompt</i> with scored prompts and exemplars	high sampling temperature for diversity
Promptbreeder[20]	seeded mutation of problem description	binary tournament	random operator out of 9 total	mutates <i>meta-prompt</i> , ICL examples
Evoprompt[18]	manual prompts and instruction induction	roulette and filter based on score	crossover and mutation	two operator variants
PE2[49]	manual or instruction induction	filter top-k prompts	<i>meta-prompt</i> with step-by-step instructions	optional task examples in <i>meta-prompt</i>
PhaseEvo[19]	manual or instruction induction	no filtering, success vector Hamming distance	instruction induction, EDA, crossover, feedback, paraphrase	alternates local, global search
PromptWizard[53]	variations from problem description	selects best prompt	critique + synthesis	improves examples after instructions
CriSPO[13]	manual initial prompt	filter top-k prompts	<i>meta-prompt</i> with critique history	self-generates critique aspects
SPRIG[54]	blank prompt	large-scale statistical resource allocation algorithms	edit-based enumeration of sentence-level operations	focus is on task-agnostic system prompts
SPO[50]	basic prompt template (e.g. CoT)	winner of pairwise evaluation	<i>meta-prompt</i> utilizing feedback from evaluation	focus is on reference-free optimization

Table 2.4: Survey of Discrete PO Methods.

We can further divide the discrete PO research into two branches, depending on whether the method optimizes instructions or examples. While some methods jointly optimize both, most research focuses on optimizing either one or optimizes them independently.

■ 2.4.1 Exemplar optimization

Exemplar optimization focuses on selecting the most effective demonstrations for ICL, a powerful technique we discussed in 2.3.1, where the LLM learns the task implicitly from labeled input-output pairs. Although underrepresented in literature[52], recent work[55][52] shows that intelligently selected exemplars often outperform optimized instructions alone and even simple optimization methods like random search can lead to significant gains across diverse tasks. This effect is further amplified when exemplar optimization is used together with instruction optimization, suggesting that the two should be co-optimized rather than treated separately[52]

Exemplar optimization is useful even for tasks without available examples with techniques like Bootstrapping[28], which utilizes inputs solved during optimization. By identifying useful and informative solutions, it allows for exemplar optimization without hand-crafted input-output pairs.

■ 2.4.2 Instruction optimization

The problem of finding the optimal instruction set can be formulated as a natural language program synthesis problem[12]. Although natural language program formulation is favorable as it represents a native interface for humans to communicate with machines, it brings its own set of problems. Compared to automatic computer program synthesis methods, the search space of natural language is much larger. This makes finding the right instructions extremely difficult.

With continuous optimization, minor perturbations of e.g. network weights generate predictably small changes in functionality. Discrete changes on the other hand often dramatically change functionality[56] and are not amenable to gradient-based optimization[57].

■ Enumeration-based approaches

First discrete PO research such as Automatic Prompt Engineer[12] (APE) relied on Monte Carlo search-inspired method based on the idea of Instruction Induction[11]. This concept works by reverse-engineering instructions from a few task samples with a *meta-prompt* template like in figure 2.9.

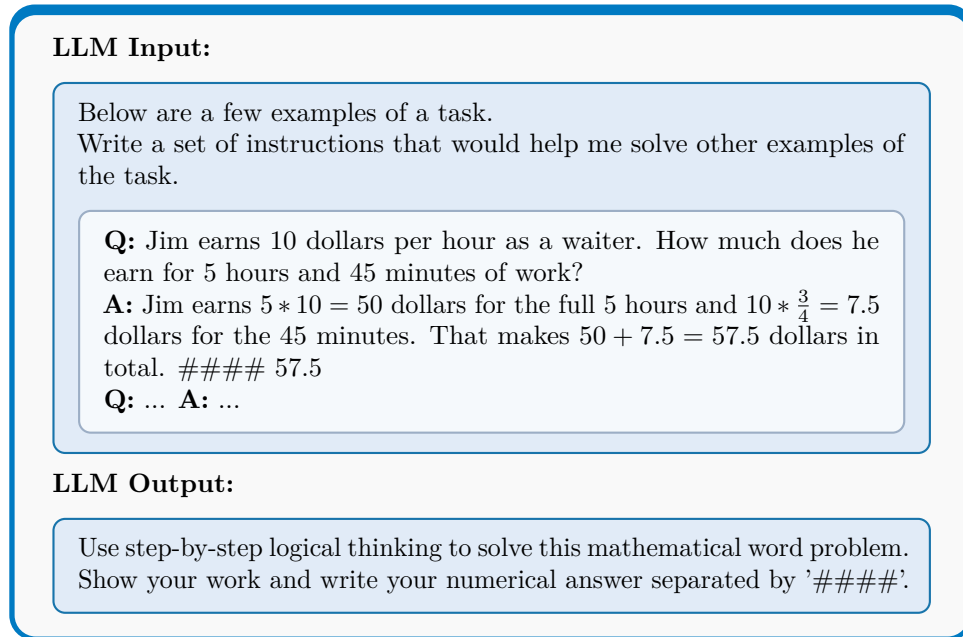
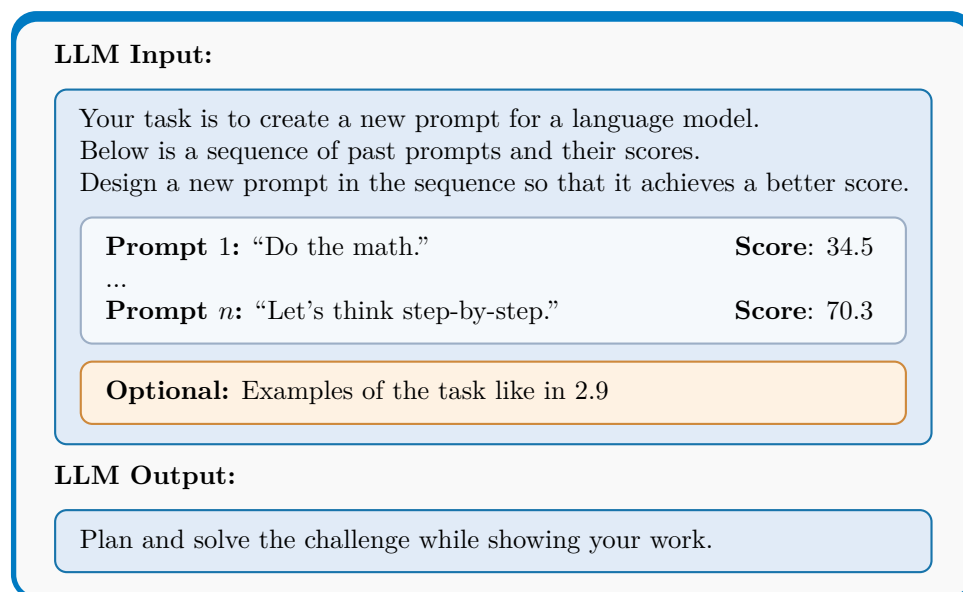


Figure 2.9: Instruction Induction

By repeating this process with a non-zero sampling temperature while varying the example set, the LLM produces a diverse set of prompts. APE then scores them on a validation set and selects the instructions with the best performance. Extending this method, iterative APE explores local space around promising prompts by paraphrasing them. This however brought marginal performance gains in comparison to the basic Instruction Induction sampling[12]. Deng et al.[57] argue that this is because "paraphrasing-then-selection" methods do not explore the prompt space systematically.

Although the research has moved onto more advanced iterative optimization methods, Instruction Induction remains one of the most common ways of initialization in literature.

The alternatives are manual initialization which either consists of a basic prompt, or expertly prompt-engineered prompts. The former poses a problem, as starting the search with high-quality instructions is essential due to the intractably large search space[55]. The latter is beneficial as it allows the



Ablations in OPRO[1] show that both the ascending order of the prompts as well showing scores is beneficial to the optimization process. Furthermore, another crucial part of the *meta-prompt* according to [1] and [58] are examples of the task, similar to the Instruction Induction *meta-prompt*. This helps the LLM better understand the task at hand. An alternative would be to include a description of the task[13].

Another branch of research, e.g. ProTeGi[51] and CriSPO[13] focus on using model feedback as a part of the optimization signal. Outputs of LLMs contain rich quality information that directly reflects prompt effectiveness[50]. Utilizing this information makes for a stronger optimization signal than just a numerical score. This is pronounced particularly for text-generation tasks, where applying prompt optimization methods based on prompt+score pairs is challenging due to the lack of effective optimization signals[13]. A single number does not capture the nuances of text and opportunities for improvement.

Pryzant et al.[51] compare the prompt critique to a gradient in the text space. The LLM can in principle, thanks to its human-like task comprehension[50], perform a sort of gradient descent by fixing the issues found by the critique.

The LLM-generated solutions to a task can be viewed as environment observations - an extrinsic information source. This is, as we discussed in 2.2, crucial for the LLM's ability to self-refine. Feedback-enriched *meta-prompts*, such as in figure 2.11, may include prompt scores as well.

The instructions in 2.10 and 2.11 are rather basic and can be endlessly improved via prompt engineering. For example, Ye et al.[49] attempt to improve on APE and ProTeGi by designing a more complete *meta-prompt*.

By extending the gradient analogy, Tang et al.[58] introduce other concepts known from traditional machine learning. First, a learning rate feature can be implemented by limiting the number of token, word or sentence edits the model can perform. Next, we can incorporate variable learning rate with strategies such as warm-up (learning rate grows in the beginning) or decay (learning rate decreases towards the end).

Another way of balancing exploration and exploitation, besides edit limit-based learning rate analogies, is tuning the LLM sampling temperature. Lower temperature encourages exploitation in the local solution space and higher temperature allows more aggressive exploration of different solutions[1].

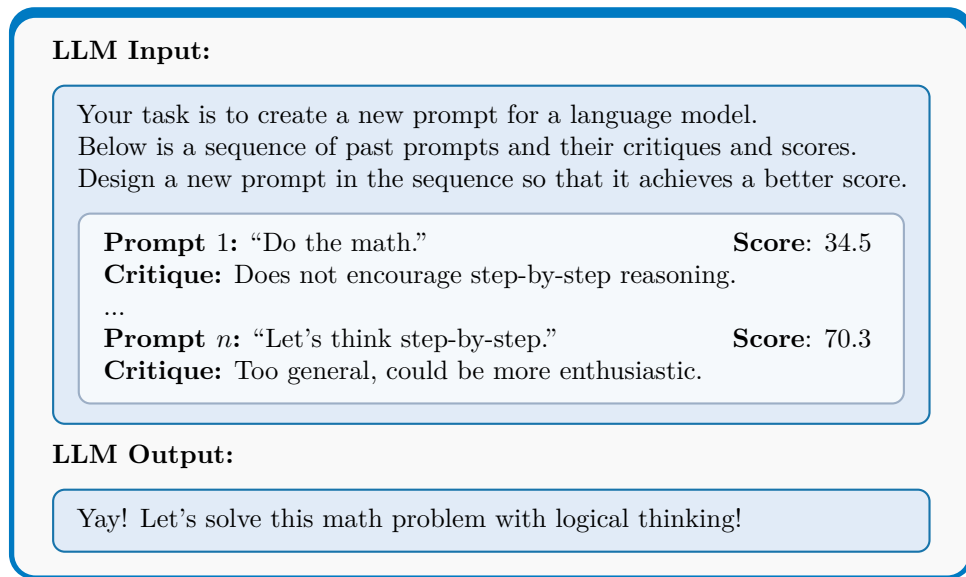


Figure 2.11: Prompt+Feedback+Score Meta-prompt

By incorporating a history of past prompts into the *meta-prompt* we can implement an analogy of momentum. This is a machine learning concept which maintains gradients from past steps and uses them in computation of the next gradient.

Including the optimization trajectory is useful but poses the problem of inflating the *meta-prompt*. This means higher costs per prompt generation, but also risks of surpassing the LLM’s context length limit. To fit into the context limit, trajectory can be summarized or retrieved based on recency, relevance or importance[58].

■ Evolution-based approaches

Evolutionary Algorithms (EAs) are a time-proven and versatile optimization method. They have been shown to be effective in search spaces with millions and billions of variables[16] by emulating the processes observed in natural evolution using crossover (sexual reproduction) and mutation (asexual reproduction) operators.

Although widely successful, EAs have been limited by the challenging nature of operator design. Developing them requires extensive manual work by experts with domain knowledge[17].

With the advent of LLMs and few-shot prompting, their ability to complete patterns can be leveraged to create a form of intelligent evolutionary crossover[16], which can be in theory truly general.

In principle, LLMs can mutate and combine any text representation that has moderate support in its training dataset and perform any genetic operator through fine-tuning or prompt engineering. The intersection between LLMs and EAs, among other traditional meta-heuristics, emerges as an exciting branch of research.

It lends itself to leverage LLM-powered EAs to improve prompts for the LLM, extending previously discussed prompt optimization methods. The seminal work in this space is EvoPrompt[18], which repurposed two EAs: Genetic Algorithm and Differential Evolution for prompt optimization. Treating the text sequences in prompts as gene sequences, EvoPrompt performs crossover and mutation on text prompts.

EA-based prompt optimization seems to demonstrate lower sensitivity to initial prompts. Although EvoPrompt utilizes some manual prompts, it achieves similar results with randomly sampled initial population as when using the best prompts[18]. On the other hand, they suffer from extremely high computational cost and slow convergence speed[19].

Methods like PhaseEvo[19] and Promptbreeder[20] strive to improve the efficiency and convergence of EA-based prompt optimization methods by supercharging them with more operators to provide balance between exploration and exploitation. Cui et al.[19] argue that the standard EA operators prioritize exploration and categorize them as global operators. To supplement them, they introduce local operators based on feedback, optimization trajectory and paraphrasing. By alternating between global and local search, PhaseEvo demonstrates better cost-efficiency compared to other EA-based prompt optimization methods[19].

In Promptbreeder[20], authors also include multiple different operators. Besides variations of previously discussed operators, they also include shuffling ICL examples and, most notably, a "hyper-prompt" which mutates the optimizer *meta-prompt* in an attempt to make the method self-referential.

A possible flaw in this method is the fact that it selects the operators randomly at each step, leading to diminished efficiency compared to a more organized approach[19], necessitating hundreds of evaluations before convergence[52].

All three discussed EA-based methods utilize Instruction Induction[11] to some extent, usually under the term "Lamarckian mutation". Besides initialization, Lamarckian mutation can be useful for adding task-relevant prompts to the population in case the optimization diverges[20].

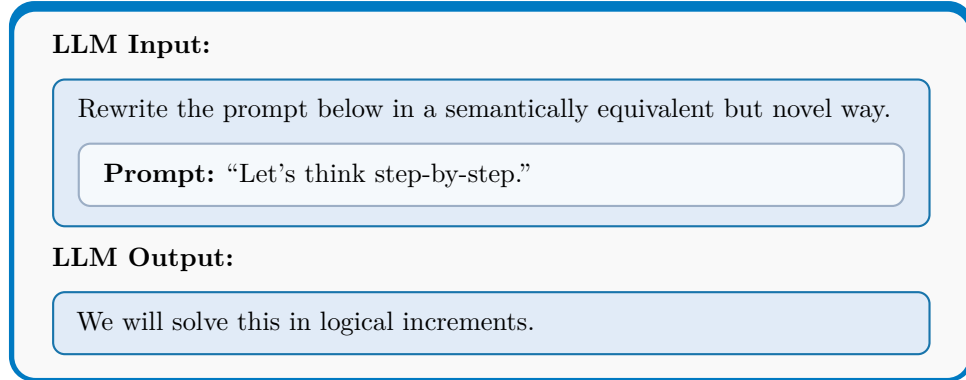


Figure 2.12: Mutation *meta-prompt*, similar to paraphrasing.

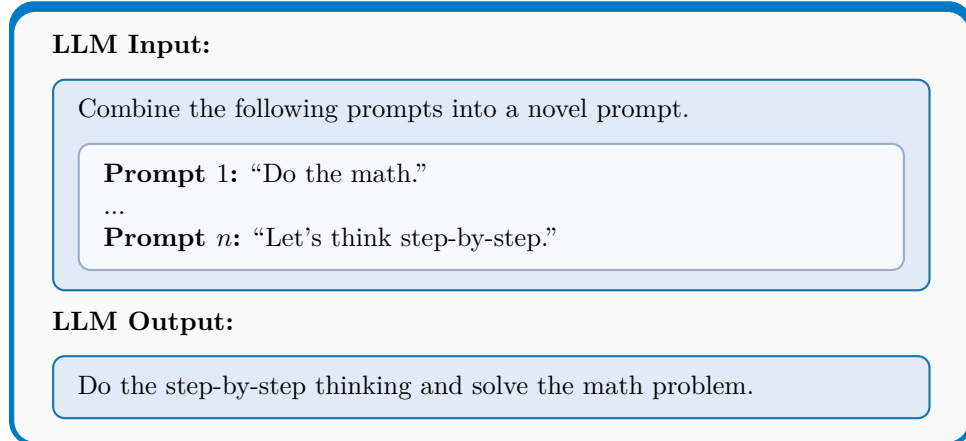


Figure 2.13: Crossover/EDA *meta-prompt*

In figures 2.12 and 2.13 the reader can find possible *meta-prompts* for a mutation and a crossover operator, respectively. By changing the meta-instructions in 2.13 and increasing n , we can shift the crossover operator into an Estimation-of-Distribution (EDA) operator. Whereas crossover aims to combine 2 or 3 prompts into 1, EDA infers the distribution of a larger number of prompts and tries to create a new prompt from that distribution.

An important factor in the implementation of the crossover and EDA operators is the way of selecting prompt specimens. In EvoPrompt[18], a roulette selection method is employed. Other methods utilize more advanced selection methods to encourage diversity.

Promptbreeder[20] filters inputs for its EDA based on their BERT embedding and PhaseEvo[19] selects parent prompts based on the Hamming distance of their "performance vectors", which hold the prompts' performance on task samples. This method ensures that the prompts that get paired with each other do not make the same mistakes.

Another important factor in the design of the operator is the ordering of input prompts. Both Promptbreeder and PhaseEvo sort the prompts in ascending order of fitness. To prevent the model from relying too much on the last prompts, the authors lie to the model by telling it the prompt are in descending order of fitness. This somewhat curbs the bias toward the later examples[20].

EAs are but one of many meta-heuristics historically used for optimization and the interactions of LLMs and meta-heuristics is poised to be a fruitful research area in the near future. In their research, Pan et al.[21] adapted and compared several meta-heuristics including Hill-Climbing, Simulated Annealing, Genetic Algorithm, Tabu Search and Harmony Search for prompt optimization.

■ Note on evaluation cost reduction

In many prompt optimization methods, prompt candidate evaluation is the most computation-intensive part of the process. Especially with score-based evaluation, it has to be performed multiple times to ensure scoring robustness[50]. To lower the evaluation costs, research adopts two approaches.

First branch, represented by ProTeGI[51] and SPRIG[54] use statistical resource allocation methods such as the Upper Confidence Bound algorithm and Successive Rejects to allocate evaluations only to promising candidates. This way, the total compute budget gets reduced. The other branch relies on ditching the numerical scoring and comparing outputs directly in a pairwise manner with a LLM judge[50].

■ 2.4.3 Multi-stage and reference-free methods

Common critique of the methods is that they are unpractical and not applicable to real-world LLM use cases. Most methods depend heavily on

external references for evaluation which are often unavailable or impractical to define, especially for open-ended tasks [50]. Xiang et al.[50] tackle this problem by using pairwise LLM-based evaluation and Zhang et al.[37] develop a gold label-free method on evaluation based on self-consistencies of different answers.

Also, as the complexity of prompt structure increases, many prompt optimization techniques are no longer applicable[14]. Schnabel et al.[14] define Symbolic Prompt Programs, representable as directed acyclic graphs. In these graphs, nodes are functions, such as `RenderText`, `GenerateResponse` or `ParseOutput` and edged are dependencies between these nodes. This representation allows for effective search with node mutators and a multitude of search algorithms.

Furthermore, modern LLM workflows interconnect various LLM and prompt instances into complex prompt programs. Most prompt optimizer approaches do not apply to these multi-stage LLM programs[55] as a whole. Optimizing each component separately is possible, but this approach ignores their mutual influence.

DSPy[28] is a Python library aiming to simplify LLM program composition with an intuitive interface. Most importantly, it allows for optimizing the whole pipeline, including exemplar optimization, instruction optimization and weight fine-tuning with the promise of declarative LLM program design without extensive prompt engineering. DSPy offers several pipeline optimization methods, like MiPROv2[55] and BetterTogether[59]. MiPROv2 generalizes OPRO[1] to multi-stage joint example and instruction optimization utilizing a surrogate Bayesian model to find the optimal configuration. With BetterTogether, Soylu et al. show that PO and fine-tuning complement each other, achieving superior performance.



Chapter 3

Implementation

3.1 Problem Formulation

Before we describe our approach and discuss implementation in detail, we will first establish mathematical notation and use it to formulate the prompt optimization task.

Let $\mathcal{T}$ be the space of character sequences. Then we will define an LLM as a stochastic mapping

$$\mathcal{M} : \mathcal{T} \rightarrow \mathcal{L}(\mathcal{T}), \quad (3.1)$$

where $\mathcal{L}(\mathcal{T})$ is a probabilistic language distribution learned during the LLM's training. This distribution is governed by the LLM's hyperparameters H , which affect its behavior.

Of particular interest is the sampling temperature $\tau \in H$ which interpolates between greedy decoding and uniform sampling. In theory, $\mathcal{M}$ is deterministic for $\tau = 0$, but in practice numerical errors still introduce variance.

We use the lower index to specify the purpose of the LLM instance. This also highlights the fact that each instance $\mathcal{M}$ can use different hyperparameters and underlying models. We will use this to differentiate between $\mathcal{M}_{\text{solve}}$ and $\mathcal{M}_{\text{optim}}$. The optimizer model $\mathcal{M}_{\text{optim}}$ will be utilized to generate and improve prompts which will be evaluated on $\mathcal{M}_{\text{solve}}$.

We consider a prompt $P \in \mathcal{I}$, where $\mathcal{I} \subseteq \mathcal{T}$ is the prompt space. When such prompt is used as an input into the LLM, it produces a completion

$$y \sim \mathcal{M}(P) \quad (3.2)$$

and $y \in \mathcal{U}$, where $\mathcal{U} \subseteq \mathcal{T}$ is the completion space. In contexts where P serves as a template for an additional query q , we will write

$$y \sim \mathcal{M}(P | q), \quad (3.3)$$

where $P | q$ denotes the result of inserting query q into a designated placeholder in P .

Let $\mathcal{P} \subseteq \mathcal{I}$ be a population of prompts. Then for a query q we define the set of completions

$$\mathcal{C}_q^{\mathcal{P}} = \{\mathcal{M}_{\text{solve}}(P | q) \mid P \in \mathcal{P}\}. \quad (3.4)$$

For convenience, we might omit the explicit population superscript and write just $\mathcal{C}_q$.

We can use LLMs to solve a general task

$$t = (q, g) \in \mathcal{D}, \quad (3.5)$$

where $\mathcal{D} \subseteq \mathcal{Q} \times \mathcal{G}$ is a dataset of query-answer pairs, $\mathcal{Q} \subseteq \mathcal{T}$ is the set of queries q and $\mathcal{G} \subseteq \mathcal{U}$ is the set of gold labels g .

We consider each dataset to have one or more assigned evaluation metrics $\mathcal{F}_{\mathcal{D}}^{\text{supervised}} : \mathcal{U} \times \mathcal{G} \rightarrow \mathbb{R}$, which scores the LLM output using the corresponding gold label. Extending the set of completions for queries from the entire dataset, we define

$$\mathcal{C}_{\mathcal{D}} = \{\mathcal{C}_q \mid q \in \mathcal{D}\} \quad (3.6)$$

For open-ended tasks, the gold label does not exist, $G = \emptyset$. To achieve effective evaluation even for such tasks, we formulate a metric based on pairwise comparisons and define

$$\mathcal{F}_{\mathcal{D}}^{\text{pairwise}} : \mathcal{Q} \times 2^{\mathcal{U}} \rightarrow \mathbb{R}, \quad (3.7)$$

which maps a query and a set of outputs to a real number. Here we use the power set notation $2^{\mathcal{U}}$ to signify the set of all possible pairs of completions $y \in \mathcal{U}$.

To generalize, we define $\mathcal{F}_{\mathcal{D}}$ which combines both $\mathcal{F}_{\mathcal{D}}^{\text{pairwise}}$ and $\mathcal{F}_{\mathcal{D}}^{\text{supervised}}$. Using this, we define the mean performance $\mathcal{E}$ of a prompt P on a dataset $\mathcal{D}$ as

$$\mathcal{E}_{\mathcal{D}}(P) = \frac{1}{\|\mathcal{D}\|} \sum_{t=(q,g) \in \mathcal{D}} \mathcal{F}(\mathcal{M}(P \mid q), t, \mathcal{C}_q). \quad (3.8)$$

For convenience, we will define the scores of the population $\mathcal{P}$ on dataset $\mathcal{D}$ as

$$\mathcal{E}_{\mathcal{D}}(\mathcal{P}) = \{\mathcal{E}_{\mathcal{D}}(P) \mid P \in \mathcal{P}\} = \{\mathcal{F}(y, g, \mathcal{C}_{\mathcal{D}}) \mid y \in \mathcal{C}_{\mathcal{D}}\}. \quad (3.9)$$

We can then formally define the problem of prompt optimization for a task dataset $\mathcal{D}$ as finding the optimal prompt

$$P^* = \operatorname{argmax}_{P \in \mathcal{I}} \mathbb{E}_{(q,g) \sim \mathcal{D}} \left[\mathcal{F}(\mathcal{M}_{\text{solve}}(P \mid q), g, \mathcal{C}_q^{\mathcal{P}}) \right]. \quad (3.10)$$

In Algorithm 1 we can see the general outline of a population-based optimization method. The initialization operator $\mathcal{O}_I$ creates an initial population of individuals $\mathcal{P}$. In each iteration, a selection operator $\mathcal{O}_S$ first selects a

Algorithm 1: General optimization loop

Input: Initialization Operator $\mathcal{O}_I$, Selection Operator $\mathcal{O}_S$, Expansion Operator $\mathcal{O}_E$, Termination Condition Φ_{stop}

Output: Optimized Population $\mathcal{P}$

Data: $\mathcal{P} \leftarrow \mathcal{O}_I$

// Initialize the population

1 **while** $\neg\Phi_{stop}(\mathcal{P})$ **do**

// Selection and Expansion Steps

2 $\mathcal{P}_{selected} \leftarrow \mathcal{O}_S(\mathcal{P})$

3 $\mathcal{P}_{expanded} \leftarrow \mathcal{O}_E(\mathcal{P}_{selected})$

4 $\mathcal{P} \leftarrow \mathcal{P}_{expanded}$ // Update the population

5 **return** $\mathcal{P}$ // Return the optimized population

portion of the population according to some criteria. These selected individuals are then used by the expansion operator $\mathcal{O}_E$ to create new individuals. This process continues until a termination condition Φ_{stop} is reached.

We apply this technique to the problem of prompt optimization by defining the aforementioned operators. Of particular interest are the initialization operator $\mathcal{O}_I$ and the expansion operator $\mathcal{O}_E$, which both need to produce new prompts. For this purpose, we utilize an LLM instance $\mathcal{M}_{\text{optim}}$ and leverage its text generation and reasoning capability.

Let $M \in \mathcal{I}$ be a *Meta-prompt*, or a prompt-generation prompt. We can now formulate generating a prompt

$$P = \mathcal{M}_{\text{optim}}(M \mid \mathcal{R}), \quad (3.11)$$

where $\mathcal{R} = \mathcal{R}(\mathcal{P}, \mathcal{C}, \mathcal{D}, \mathcal{E}_{\mathcal{D}}(\mathcal{P}))$ is a retrieval function that selects data from the current population, past generations, dataset samples and population scores. By changing the *Meta-prompt* and the retrieval function $\mathcal{R}$, a variety of possible operators $\mathcal{O}_I$ and $\mathcal{O}_E$ can be defined, thus shaping the prompt optimization process.

■ 3.2 Inference framework

Taking inspiration from DSPy[28], we first implement a simple LLM-calling framework capable of invoking several selected inference strategies. Motivations for this are twofold:

1. DSPy is a young and ambitious project aiming at simplifying LLM pipeline design and optimization. As we focus on single-stage prompt program optimization, this capability is not useful for our work. Furthermore, due to the framework’s infancy, it lacks proper documentation and sometimes exhibits unexpected behavior.
2. Implementing the prompting techniques discussed in 2.2 provides further insight into their workings and performance.

■ 3.2.1 Structured generation

Following current research trends[45], we build our inference framework around a structured JSON template, or a **Signature**. The **Signature** structure consists of input and output fields and additional instructions. These fields are populated by a **Field** data structure. Of particular interest are the output fields, which hold the output name, desired type and optional description.

Interactions with LLMs in a structured format benefit from better predictability. By implementing the **Signature** structure, we can use LLMs as we would a function in any programming language. Functions in programming languages also have functions signatures which specify input and output names and types.

When employing good naming practices the model can often deduce the task only by looking at output names and types. Consider the simple **Signature** in figure 3.1, which implicitly instructs the LLM to return a word with a meaning opposite to the provided input.

For more complex tasks, filling the output descriptions or even adding explicit instructions is necessary. In figure 3.2, notice that it is possible to specify multiple inputs and outputs, which are then generated in the order given.

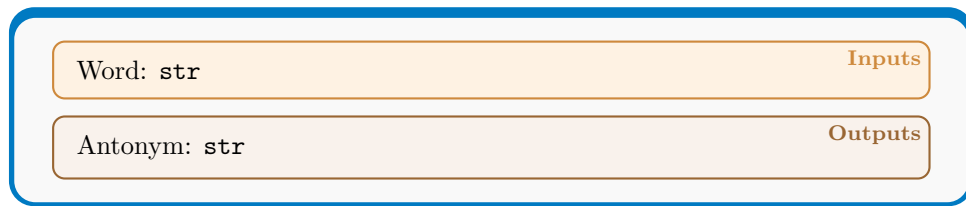


Figure 3.1: Simple Signature

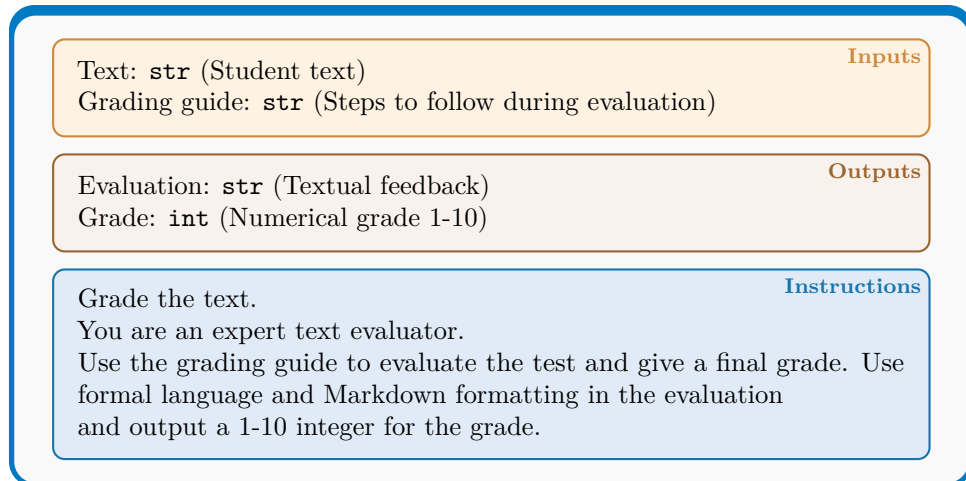


Figure 3.2: Complex Signature.

We will maintain this formatting style whenever showing a **Signature** structure in the future: orange inputs, brown outputs and optional blue instructions.

Sufficiently large instruction-tuned LLMs are usually good at reliably producing JSON output. For smaller models or more complex output structures, it might be necessary to use some form of constrained generation as discussed in 2.2. A JSON schema could be constructed automatically from the **Signature** and passed into a parser-based sampler. However, this is not necessary for our use-case and the only safeguard we implement is repeated generation in case of a parsing error.

3.2.2 Predict method

To facilitate **Signature**-powered generation, we implement a **predict** method that involves 1) prepending a developer prompt to the messages, and 2) parsing of **Signature** outputs.

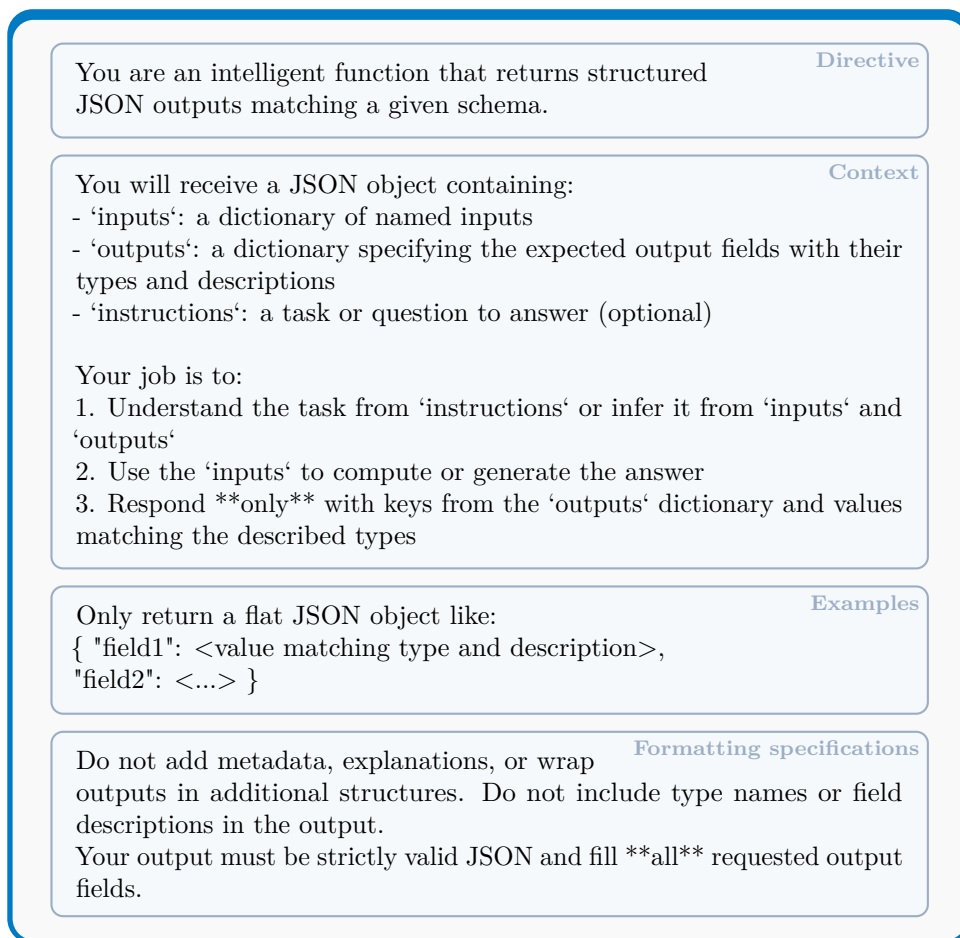


Figure 3.3: Predict method developer prompt with highlighted prompt sections: directive, context, examples and format specifications.

The developer prompt has to clearly explain to the LLM how to work with the JSON-based **Signature**. In figure 3.3 notice the sections of the prompt following prompt engineering principles outlined in 2.3. The directive in this case slightly overlaps with the context and includes a memetic proxy by telling the model it is “an intelligent function”.

First, the directive states the task, then further context is added about the **Signature** data structure and the task. Next, notice the example showing the proper output. Finally, few more clarifying instructions about the output format are added. In experiments, this prompt is successful in incentivizing parsable outputs adhering to the **Signature** specifications.

Parsing the output presents some challenges as the LLM sometimes wraps the JSON output into a Markdown code block or uses inconsistent escape sequences. We implement a simple parser based on regular expressions

that is able to parse the majority of outputs. In case of a parsing issue or model failure, such as getting stuck in a generation loop, we add a repeated generation feature.

3.2.3 Inference techniques implementation

Leveraging the `predict` method and the modular **Signature**-based interface, we implement a suite of inference-time prompting techniques. Each technique is realized through systematic modifications of the **Signature** fields, changing the developer prompt and the chaining of multiple generation steps and function calls. This design allows for modularity and reuse while preserving transparency. We implement the following methods.

1. **Chain-of-thought**[29]: Prepends a reasoning field to the **Signature** outputs which forms a scratch pad for the LLM.
2. **Chain-of-thought with Self-consistency**[36]: Multiple CoT generations with majority-voting.
3. **ReAct**[27]: Adding tools allows the LLM to interleave thoughts and action steps.
4. **Program-of-thought**[26]: Two-stage CoT with Python-code execution
5. **Reflexion**[42]: After an initial generation, the model is prompted to self-critique and revise its output.
6. **Tree of Thoughts**[31]: The problem is first decomposed and each step is expanded, forming a thought tree, which is then traversed with BFS or DFS.

These techniques are however not the main focus of this bachelor's thesis, and we proceed without further discussion or evaluation. In our prompt optimization method, we will utilize only the basic Chain-of-thought module.

3.3 Datasets

In this section, we discuss choosing datasets for testing our method and comparing various prompt optimization approaches. While searching available datasets, we focus on the following criteria:

1. **Output complexity:** We focus on more complex outputs. Specifically, datasets with multiple-choice or Yes/No answers are omitted. This disqualifies commonly used datasets as MMLU or BigBenchHard.
2. **Contamination:** Recently, researchers have expressed concern[15] whether benchmarks are reliable evaluations of models as they might appear in their training data. We omit most common datasets, such as GSM8k[60], which has been shown to have inflated scores for some models[61].
3. **Output verification:** We prefer to use simple automatic verification rather than using LLM-as-a-judge, which has been shown to be biased in some circumstances[62]. Neither do we use human feedback, which defeats the purpose of automatic prompt optimization.
4. **Difficulty:** We omit tasks where models already have near-perfect score.
5. **Benefit from non-trivial instruction:** We focus on tasks where helpful hints and step-by-step tutorial-like instructions might increase the probability of successful solution.

We now list the datasets that we will use for evaluation and explain why they were chosen.

■ 3.3.1 Livebench

The Livebench[15] dataset is very recent and has been created with the issue of data contamination in mind. It also addresses the issues of LLM-as-a-judge verification and all its categories can be verified automatically. It is also very challenging, with top models achieving 65% accuracy[15].

Out of the tasks available in Livebench, we select the **Connections** task from the **Language** subset. This task consists of sorting given words into non-trivial groups of four based on semantics, phonetics and other features. An ideal prompt would attempt to list multiple possible aspects based on which the words can be sorted and also include a helpful example.

■ 3.3.2 Code Contests

Programming puzzles are a difficult and easily verifiable task. Although `CodeContests`[63] is an older dataset, we anticipate this dataset presents reduced contamination risks compared to datasets with simpler outputs. With LLM-powered coding assistants on the rise, we feel this is a relevant application area for our method.

■ 3.3.3 Sequences

We design a small but challenging dataset based on predicting the next number in an integer sequence. Each sequence is created according to a formula with randomly selected coefficients. The formulas fall into several categories, for example

- **Linear with modulo:** $s(i) = \text{mod}_q(a_1i + b_1)$
- **Sum:** $s(i) = \sum_{j=0}^{i-1} a_1j + b_1$
- **Alternating:** $s(i) = a_1i + a_2i(-1)^i$.

This tests the model’s ability to 1. detect and understand patterns and 2. systematically perform simple arithmetic. In practice, we will optimize just for a single sequence category and observe whether the optimizer evolves a prompt with a tutorial for the specific sequence category. Experiments showed that the **Alternating** class of sequences has a good difficulty balance, and we will use it for evaluation.

■ 3.4 Evaluation Metrics

The evaluation metric defines the optimization goal and thus forms its central component. Most evaluation metrics are task-specific and divisible into two categories based on whether they are used in a supervised or self-supervised context.

3.4.1 Metrics for Supervised Optimization

Supervised optimization is supported by gold labels and its underlying metrics all perform comparisons between the results and the gold labels. These include classification metrics like accuracy or Hamming Loss, regression metrics like Mean Squared Error, and many others.

All three main benchmarks that we will use (**Connections**, **CodeContests**, **Sequences**) fall into this category. For each benchmark we use a simple accuracy metric. Given a dataset $\mathcal{D}$ and questions q and gold labels g , $(q, g) \in \mathcal{D}$:

$$\textbf{Connections: } \mathcal{F}_{\mathcal{D}_{\text{Conn}}}(q, g) = \text{Overlap}(\text{Groups}(q), \text{Groups}(g)) \quad (3.12)$$

$$\begin{aligned} \textbf{CodeContests: } \mathcal{F}_{\mathcal{D}_{\text{Code}}}(q, g) = & \text{FinishesExecution}(q) \\ & + \text{PassesAllCases}(q, g) \end{aligned} \quad (3.13)$$

$$\textbf{Sequences: } \mathcal{F}_{\mathcal{D}_{\text{Seq}}}(q, g) = \text{Equals}(q, g) \quad (3.14)$$

3.4.2 Metrics for Self-Supervised Optimization

In self-supervised contexts, metrics are usually based on reward models pretrained on human preference or environment data. To allow our method to be applied to gold label-free problems, we turn to LLM-based direct pairwise comparisons. Given a dataset $\mathcal{D}$ with queries q , output y produced by prompt $P \in \mathcal{P}$ and a set of completions $\mathcal{C}_q$ for each query.

$$\mathcal{F}_{\mathcal{D}}^{\text{pairwise}}(q, y, \mathcal{C}_q) = \text{WinRate}(\{\text{Compare}(q, y, c) \mid c \in \mathcal{C}_q\}). \quad (3.15)$$

In practice, we combine the output comparison with comparing the output's respective prompts. These comparisons are then used as optimization signals in the **Feedback** operator.

3.5 Optimization Framework

Although our first implementation attempt utilized an evolutionary algorithm, we will use a basic population-based hill-climber algorithm. This design decision has several reasons.

1. Most PO research uses a hill-climber architecture.
2. Prompt optimization based on EAs suffer from slow convergence compared to state-of-the-art hill-climber PO[50].
3. PO is complex as it is, and more complicated architectures only introduce more hyperparameters.

Algorithm 2: Prompt Optimization Hill-Climber

Input: Dataset $\mathcal{D}$, Population size S , Iteration count I , Batch size B
Output: Optimized Prompts $\mathcal{P}^*$

```

1  $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{dev}}, \mathcal{D}_{\text{test}} \leftarrow \text{Split}(\mathcal{D})$  // Generate training splits
2  $\mathcal{P} \leftarrow \text{InstructionInduction}(\mathcal{D}_{\text{train}})$  // Induce initial prompts
3  $i \leftarrow 0$  // Initialize iteration count
4  $\mathcal{C} \leftarrow \{\}$  // Initialize solutions
5  $\mathcal{E} \leftarrow \{\}$  // Initialize scores
6  $\mathcal{A} \leftarrow \mathcal{P}$  // All prompts
7 while  $i < I$  do
8    $Q, G \leftarrow \text{RandomSample}(\mathcal{D}_{\text{dev}}, B)$ 
9    $\mathcal{C} \leftarrow \{\mathcal{C}_q^{\mathcal{P}} \mid q \in Q\}$ 
10   $\mathcal{E} \leftarrow \text{Evaluate}(\mathcal{C}, G)$ 
11   $\mathcal{P} \leftarrow \text{Selection}(\mathcal{P}, \mathcal{E})$  // Pruning
12   $\mathcal{P} \leftarrow \text{Expand}(\mathcal{P}, \mathcal{C}, \mathcal{E}, \mathcal{D}_{\text{train}})$ 
13   $\mathcal{A} \leftarrow \mathcal{A} \cup \mathcal{P}$ 
14  $P^* \leftarrow \underset{P \in \mathcal{A}}{\text{argmax}}(\mathcal{E}_{\mathcal{D}_{\text{test}}}(P))$ 
15 return  $P^*$ 

```

In Algorithm 2 we iterate on the general algorithm 1. We will discuss the design of functions used in 2 in following sections.

- **Expand:** The Expand function can be filled with various expansion operators, of which InstructionInduction is a special case.
- **Evaluate:** Evaluating and identifying the most promising prompts is handled by the Evaluate operator, which uses task-specific automatic evaluation or LLM-feedback.
- **Selection:** The Selection operator prunes the population and should maintain only the most promising and diverse prompts for the next expansion.

■ 3.5.1 Expansion Operator Design

Expansion operators' job is extending the optimization population with new prompts. Remember notation from 3.11:

$$P = \mathcal{M}_{\text{optim}}(M \mid \mathcal{R}).$$

Notice the use of $\mathcal{M}_{\text{optim}}$, which utilizes non-zero sampling temperature $\tau > 0$ to encourage output diversity. Evidently the prompt generation task can be separated into two independent problems: 1. crafting the optimal *meta-prompt* M and 2. designing a data retrieval function $\mathcal{R} = \mathcal{R}(\mathcal{P}, \mathcal{C}, \mathcal{D}, \mathcal{E})$. The operators' design should address the following challenges:

1. **Loss of generality:** When using task samples $(q, g) \in \mathcal{D}$, the model $\mathcal{M}_{\text{optim}}$ might focus on a single query q and thus fail to generate general instructions.
2. **Loss of diversity:** Even for $\mathcal{M}_{\text{optim}}$ with $\tau > 0$, the resulting prompts can be very similar and fail to explore the prompt space $\mathcal{I}$. This ties into a broader exploration vs. exploitation balance issue.
3. **Lack of optimization signal:** Research[13][50] suggests that $\mathcal{M}_{\text{optim}}$ can make use of feedback on prompts' outputs and that these textual signals are more effective than numerical scores.
4. **Out of distribution *meta-prompt*:** Prompt engineering is a novel research area and does not have a substantial support in the LLM's training corpus. The *meta-prompt* M thus has to be carefully constructed to help the model output relevant prompts.

We now discuss the design of each prompt generation operator and display their signatures and *meta-prompts*. Note that all operators are ultimately used in a CoT context, where a **reasoning** field is prepended to each signature's outputs.

■ Lamarckian

Instruction Induction[11] is used by many PO methods and often referred to as **Lamarckian Mutation**. We will adopt this terminology from now on and design our **Lamarckian** operator. Design of its meta-prompt, shown in figure 3.4, takes into account the design challenges mentioned earlier by 1. warning

the LLM to be general and not to focus on a single example, 2. clearly states the problem using a directive and formatting specifications.

The problem with diversity still persists, and we consider two approaches to solving it. We can increase the model’s creativity by increasing its sampling temperature. Another approach is to use some kind of *seed*, for example a *persona*. We experiment with using personas from PersonaHub[64]. Authors of this paper argued that seeding generation with the persona helps with creating novel synthetic data.

For the data retrieval part, **Lamarckian** utilizes only examples of the datasets. We randomly sample N examples from a separate training split. So

$$\mathcal{R}_L(\mathcal{D}) = \text{RandomSample}(\mathcal{D}_{\text{train}}, N) \quad (3.16)$$

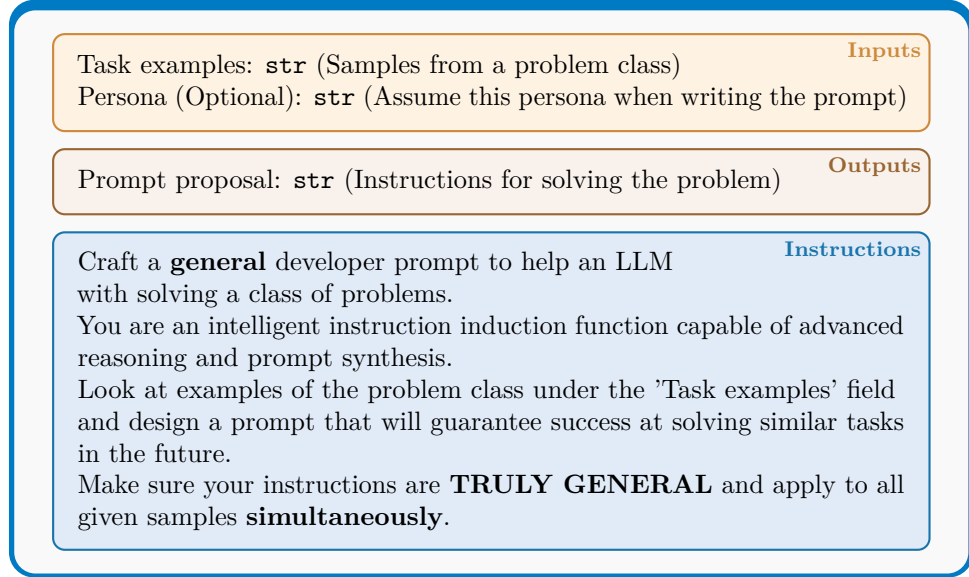


Figure 3.4: Lamarckian Signature. Formatting specifications trimmed.

Iterative

The **Iterative** operator is one of the most common and simplest operators. It uses a sequence of prompts and their scores in ascending order. The hope is for the LLM to deduce the optimization direction by looking at the differences in the prompts and incite it to continue the pattern.

Although some research[1] only uses the top prompts, we opt for a roulette selection method and sort to prompts by score in ascending order. The number N is a hyperparameter dictating how many prompts to sample. We define the retrieval function as

$$\mathcal{R}_I(\mathcal{P}, \mathcal{E}) = \text{SortByScore}(\text{RouletteSampling}(\mathcal{P}, \mathcal{E}, N), \mathcal{E}) \quad (3.17)$$

Other methods[58] additionally augment the *meta-prompt* with task samples, similar to the **Lamarckian** operator.

In the *meta-prompt*, we instruct the LLM to try to follow the sequence. Also, we specifically say to 'craft a new prompt' as opposed to 'improve a prompt' to incite more novelty. For formatting, we use the same instruction set as in the **Lamarckian**.

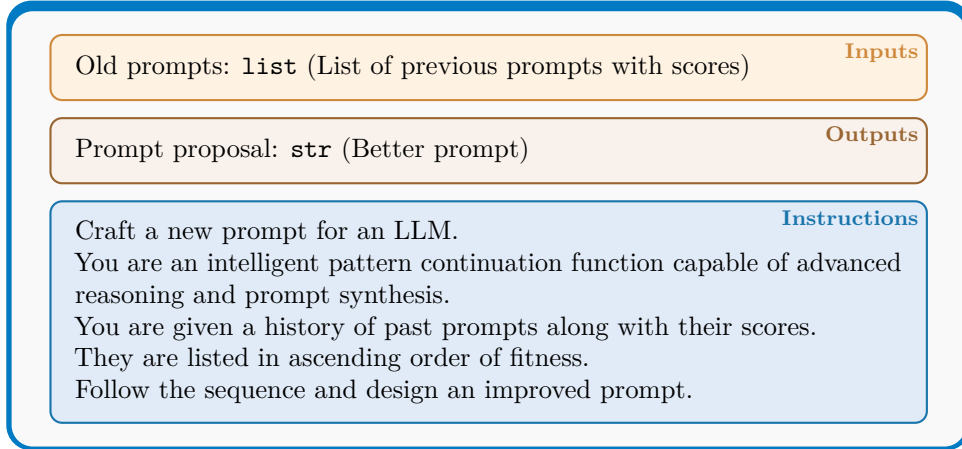


Figure 3.5: Iterative Signature. Formatting specifications trimmed.

Reflective

Recent PO literature[50] shifts to using LLM outputs as optimization signals and argues that utilizing only numerical signals is ineffective. To address this,

we design an exploitative operator, which aims to fix faults in the prompt by analyzing its failed attempt at a task sample.

To achieve this, a more complex **Signature** is utilized. Its outputs guide the LLM to first critique the original prompt and then improve it. Instructions are more complete with a step-by-step guide which explains the task clearly. Note that 1. now we use "improve" wording, 2. we stress to only alter the prompt *slightly*. This is done due to frequent observation of the model just creating an entirely different prompt only applicable to the single example task. For formatting, we use the same instructions as in previous *meta-prompts*.

In $\mathcal{R}$, we want to select the worst possible attempt. This means we optimize "from the bottom up" and try to bootstrap the worst prompts. The retrieval function is

$$\mathcal{R}_R(\mathcal{P}, \mathcal{C}, \mathcal{D}, \mathcal{E}) = \text{FindWorstAttemptData}(\mathcal{P}, \mathcal{C}, \mathcal{E}, \mathcal{D}) \quad (3.18)$$

The function **FindWorstAttemptData** finds the worst attempt overall. This is done with the goal of guaranteeing an attempt with score 0. We first find the prompt with the worst worst-case performance and then fetch data about its worst attempt - task question, reasoning chain (solution) and the prompt itself.

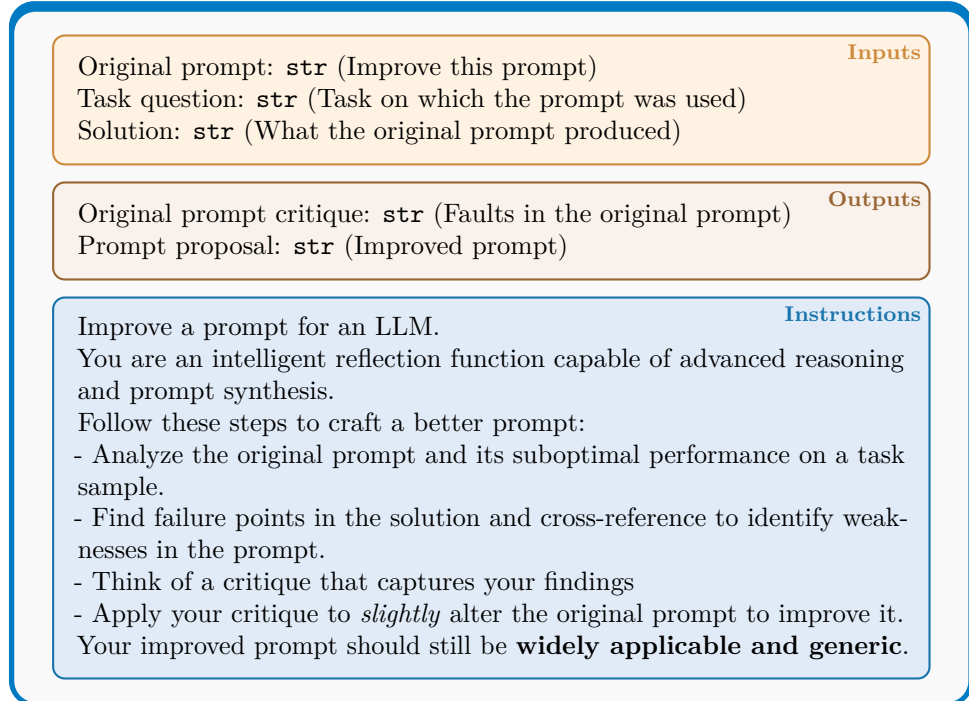


Figure 3.6: Reflective Signature. Formatting specifications trimmed.

Feedback

As we mentioned earlier, the **Feedback** operator is suitable for use in self-supervised settings. It leverages reasoning traces from pairwise LLM-based comparisons, discussed in 3.4.2. Let $\mathcal{E}_{\text{comp}}$ hold textual comparisons of each prompt and their attempts and $P_{\text{base}} = \text{RandomSample}(\mathcal{P})$. Then

$$\mathcal{R}_F(\mathcal{P}, \mathcal{E}_{\text{comp}}) = \{P_{\text{base}}, \text{GetComparisons}(P_{\text{base}}, \mathcal{E}_{\text{comp}})\} \quad (3.19)$$

In the *meta-prompt*, we frame the task as critique synthesis and use "improve" wording to guide the LLM to start from the base prompt. We also explain that each comparison has a different verdict and the base prompt might not always be the winner. For the formatting guide, we use the same instructions as in the previous operators. In figure 3.8, we show an example of a comparison between two outputs and their prompts. This is a by-product of gold label-free evaluation.

For large populations or tasks producing long prompts, we might run into issues with LLM context window length. However for our purpose, modern LLMs provide more than sufficient context limits.

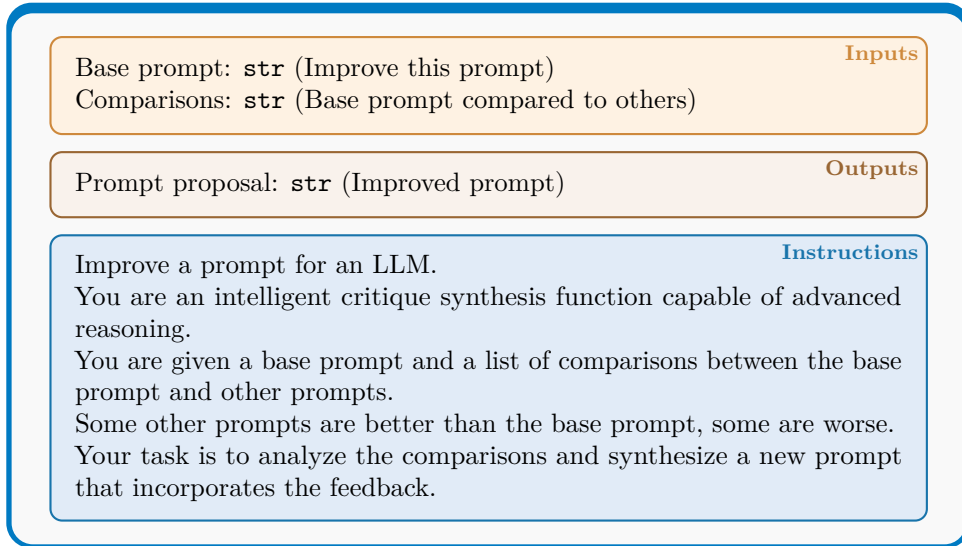


Figure 3.7: Feedback Signature. Formatting specifications trimmed.

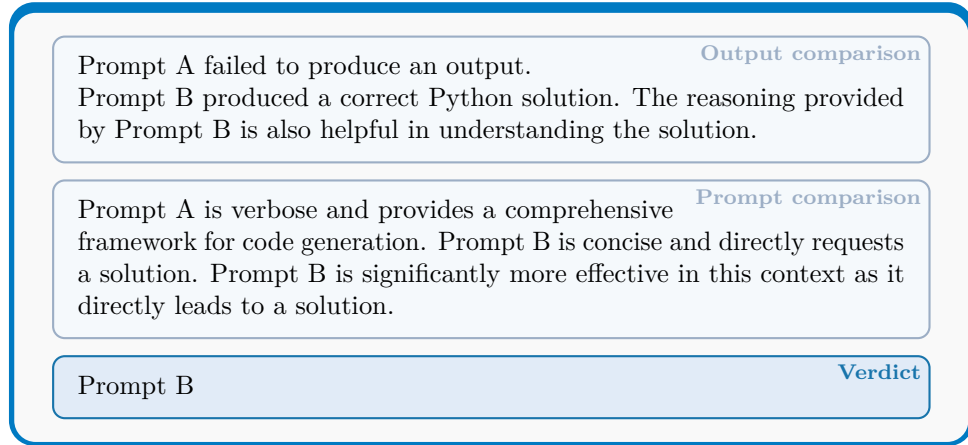


Figure 3.8: Example of a comparison of two prompts for CodeContests

■ Paraphrase

To serve as another baseline for other operators, we implement a simple **Paraphrase** operator. This operator performs random search in the prompt space by changing the wording and structure of a prompt. The prompt is selected via the retrieval function

$$\mathcal{R}_P(\mathcal{P}, \mathcal{E}) = \text{RouletteSampling}(\mathcal{P}, \mathcal{E}). \quad (3.20)$$

This method uses no optimization signal or improvement instructions and relies on pure chance of finding a more potent prompt.

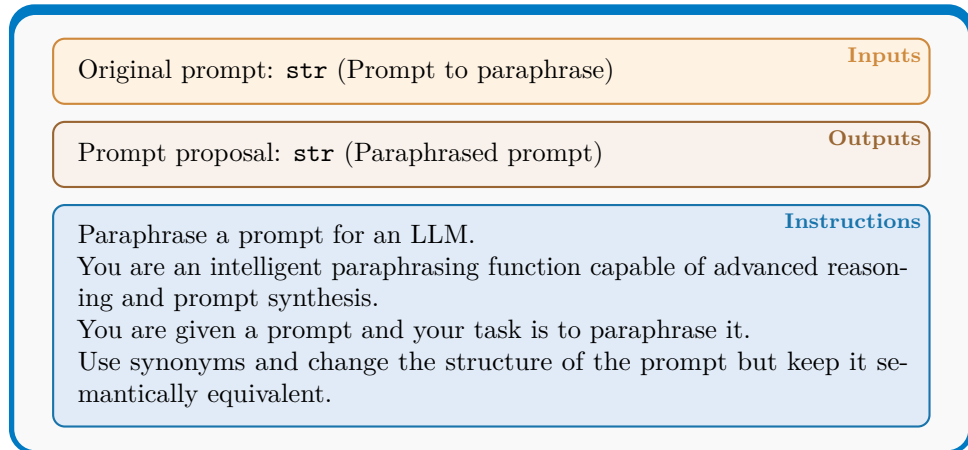


Figure 3.9: Paraphrase Signature. Formatting specifications trimmed.

In figure 3.10 we show the formatting specification part, which is identical for all prompt generation *meta-prompts*.

Use Markdown formatting in your final answer to indicate bullet points and whatever else necessary.
 As a placeholder for the task question, '<INSERT TASK QUESTION HERE>' should be used exactly ONCE.
 In the final answer, do not include a title or any additional data, just the prompt.

Figure 3.10: Formatting specifications, which are identical for all operators

■ 3.5.2 Selection Operator

At the start of each optimization step, we select n_{continue} prompts to continue in the process and purge the rest. To achieve better prompt diversity, a method based on edit distance is used.

This method, outlined in Algorithm 3, removes the closest prompt for each prompt, starting from the best prompts. This ensures that performant prompts are kept and their worse-performing duplicates are deleted. We opt to use edit distance instead of semantic similarity, like BERT embeddings.

Algorithm 3: Purge Duplicates

Input: Population $\mathcal{P}$, Pruning factor f_{prune}
Output: Pruned population $\mathcal{P}_{\text{pruned}}$

- 1 $\mathcal{P}_{\text{sorted}} \leftarrow \text{SortByScoreDescending}(\mathcal{P}, \mathcal{E})$
- 2 $n_{\text{continue}} \leftarrow |\mathcal{P}|(1 - f_{\text{prune}})$
- 3 $i \leftarrow 0$
- 4 **while** $i < n_{\text{continue}}$ **do**
- 5 $P_{\text{select}} \leftarrow \text{Get}(\mathcal{P}_{\text{sorted}}, i)$ // Get i-th best
- 6 $P_{\text{purge}} \leftarrow \underset{P \in \mathcal{P} | P \neq P_{\text{select}}}{\text{argmax}} \text{ LevenshteinRatio}(P, P_{\text{select}})$
- 7 $\text{Remove}(\mathcal{P}, P_{\text{purge}})$
- 8 $\mathcal{P}_{\text{pruned}} \leftarrow \mathcal{P}$
- 9 **return** $\mathcal{P}$



Chapter 4

Experiments

■ 4.1 Supervised Optimization Operator Evaluation

In our main experiment, we compared the effectivity of 4 optimization operators on 3 diverse gold-labeled datasets. We compared the results and costs of each method against each other and a strong Instruction Induction baseline.

All our experiments were conducted through the **OpenRouter** API, which offers many LLMs from different providers. We differentiate between optimizer LLM $\mathcal{M}_{\text{optim}}$ and $\mathcal{M}_{\text{solve}}$. Both use the medium model **Google Gemma 3 27B**[65]. To encourage diversity, $\mathcal{M}_{\text{optim}}$ works with sampling temperature $\tau = 0.75$. Bigger sampling temperature was observed to diminish the model’s ability to follow structured output specifications. To keep prompt testing as deterministic as possible, we initialize $\mathcal{M}_{\text{solve}}$ with $\tau = 0.0$.

■ 4.1.1 Experiment setup

We tested our method against 3 datasets, **CodeContests**, **Connections** and **Sequences**, which have 30 samples each and form 3 equal splits $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{dev}}$ and $\mathcal{D}_{\text{test}}$. Each of the 4 operators - **Reflective**, **Iterative**, **Feedback** and **Paraphrase** - is tested on all three datasets for 3 repetitions to alleviate randomness in the results.

To create a strong baseline, we started by creating 50 prompts with Instruction Induction through the **Lamarckian** operator using **Persona**[64] seeding. These prompts form $\mathcal{P}_{\text{baseline}}$. Examples are sampled from $\mathcal{D}_{\text{train}}$ with the whole 10 sample split being used for **Connections** and **Sequences** and 3 samples for **CodeContests**. This is because code input/output pairs are a lot longer, and we observed a greater likelihood of failure to follow output structure for more samples.

These 50 prompts were evaluated on $\mathcal{D}_{\text{test}}$ and the fourth quintile (prompts 11-20 when ranked by test score) was used as the initial population $\mathcal{P}_{\text{init}}$ for the optimizer. We used population size $S = 10$, iteration count $I = 10$, batch size $B = 3$ and pruning factor $f_{\text{prune}} = 0.5$. This means that each step produced 5 new prompts and 50 prompts overall. We selected these parameters to make the optimization process resources *roughly* comparable to the Instruction Induction baseline generation.

4.1.2 Experiment results

In tables 4.1, 4.2 and 4.3 we show the results of our main experiment on **CodeContests**, **Connections** and **Sequences** respectively. In the row marked HB, we present the maximum score achieved by prompts $P \in \mathcal{P}_{\text{baseline}}$, forming a hard baseline (HB). The next row, marked SB, shows the best score out of the initializing prompts $P \in \mathcal{P}_{\text{init}}$, which forms a soft baseline (SB). The following rows show the best score of each step s , which is calculated as an average over all three experiment repetitions $e \in \{1, 2, 3\}$:

$$\frac{1}{3} \sum_{e=1}^3 \text{Max}(\mathcal{E}_{\mathcal{D}_{\text{test}}}(\mathcal{P}_s^e)), \quad (4.1)$$

where $\mathcal{P}_s^e$ signifies the population at step s of experiment repetition e . For each operator, the best step score is marked in **bold** and the best score overall is also underlined. Cells with values which surpass the soft baseline are shaded in green and those which fall short are in red. We also include the average step score in bracketed smaller script, defined by

$$\frac{1}{3} \sum_{e=1}^3 \text{Mean}(\mathcal{E}_{\mathcal{D}_{\text{test}}}(\mathcal{P}_s^e)). \quad (4.2)$$

Figures 4.1, 4.2 and 4.3 show the corresponding graphs with a gray dotted line marking the soft baseline and an orange solid line marking the hard baseline.

Somewhat counter-intuitively, we observe that the simpler operators, **Iterative** and **Paraphrase**, consistently outperformed the others over all datasets and are the only ones which outperformed the hard baseline at any point.

The most complicated operator, **Feedback**, which uses binary output and prompt comparisons, exhibited the worst performance overall and at one point fell under the soft baseline for every task. This also happened to the **Reflective** operator on the **Connections** task.

Improvements over hard baseline are marginal, and we have to conclude that the optimization did not produce prompts that would help the model solve the task consistently. However, we observe significant gains over the soft baseline, which was used to initialize the experiment.

In figure 4.4 we show the progressive score change relative to the soft baseline over all experiments. This graphic again illustrates that the **Feedback** operator consistently downgraded performance.

STEP	REFLECTIVE	ITERATIVE	FEEDBACK	PARAPHRASE
HB	0.26	0.26	0.26	0.26
SB	0.16	0.16	0.16	0.16
1	0.23 (0.15)	0.17 (0.12)	0.26 (0.15)	0.22 (0.14)
2	0.22 (0.12)	0.25 (0.16)	0.11 (0.06)	0.22 (0.14)
3	0.23 (0.15)	0.19 (0.12)	0.16 (0.07)	0.19 (0.14)
4	0.18 (0.14)	0.27 (0.15)	0.09 (0.05)	0.19 (0.14)
5	0.23 (0.17)	0.16 (0.11)	0.09 (0.05)	0.22 (0.14)
6	0.23 (0.17)	0.18 (0.11)	0.08 (0.04)	0.23 (0.14)
7	0.25 (0.16)	0.25 (0.13)	0.05 (0.04)	0.19 (0.12)
8	0.15 (0.12)	0.23 (0.16)	0.08 (0.05)	0.17 (0.11)
9	0.23 (0.14)	0.19 (0.11)	0.15 (0.07)	0.17 (0.12)
10	0.24 (0.13)	0.25 (0.13)	0.08 (0.04)	0.23 (0.15)

Table 4.1: Results for CodeContests. Listed values are calculated using equations 4.1 and 4.2 according to the metric function in equation 3.13.

STEP	REFLECTIVE	ITERATIVE	FEEDBACK	PARAPHRASE
HB	0.33	0.33	0.33	0.33
SB	0.20	0.20	0.20	0.20
1	0.29 (0.15)	0.27 (0.16)	0.20 (0.15)	0.31 (0.21)
2	0.20 (0.13)	0.29 (0.18)	0.24 (0.14)	0.32 (0.20)
3	0.20 (0.14)	0.27 (0.17)	0.19 (0.14)	0.29 (0.21)
4	0.18 (0.04)	0.26 (0.17)	0.09 (0.06)	0.28 (0.19)
5	0.16 (0.08)	0.27 (0.18)	0.14 (0.09)	0.31 (0.20)
6	0.17 (0.09)	0.27 (0.19)	0.13 (0.08)	0.32 (0.21)
7	0.16 (0.08)	0.29 (0.19)	0.10 (0.04)	0.30 (0.19)
8	0.16 (0.08)	0.28 (0.19)	0.07 (0.04)	0.33 (0.23)
9	0.16 (0.07)	0.27 (0.20)	0.05 (0.03)	0.28 (0.22)
10	0.16 (0.07)	0.26 (0.16)	0.07 (0.03)	0.35 (0.25)

Table 4.2: Results for Connections. Listed values are calculated using equations 4.1 and 4.2 according to the metric function in equation 3.12.

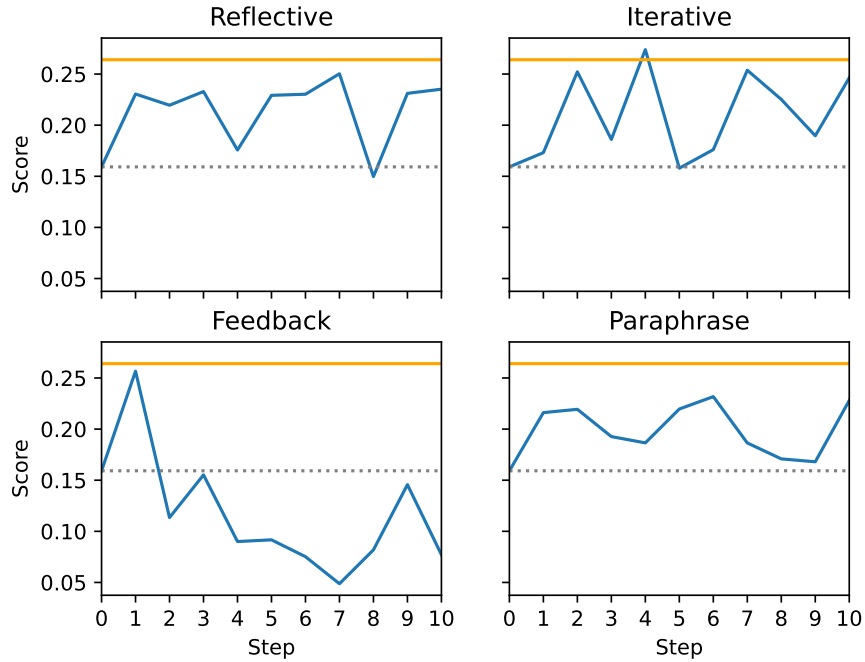


Figure 4.1: Optimization progression for the CodeContests dataset.

We also graph token usage during optimization in figure 4.5. Due to the immense requirements of the **Feedback** operator, a logarithmic scale is used. While other operators need around 200 thousand tokens for a complete optimizer routine, **Feedback** needs almost 4 million. For our model, this translates to about 0.03 USD and 0.60 USD respectively.

4.1.3 Discussion

Contrary to expectations, operators utilizing textual feedback signals were outperformed by simpler operators both on raw performance and on the ratio of performance to computation resources. The most striking was the failure of the **Feedback** operator. This operator, which needed almost 20x the tokens as the other operators, proposes new prompts using a history of comparisons between a base prompt and other prompts and their outputs.

We now discuss several possible explanations for the failure of the **Feedback** and **Reflective** operators.

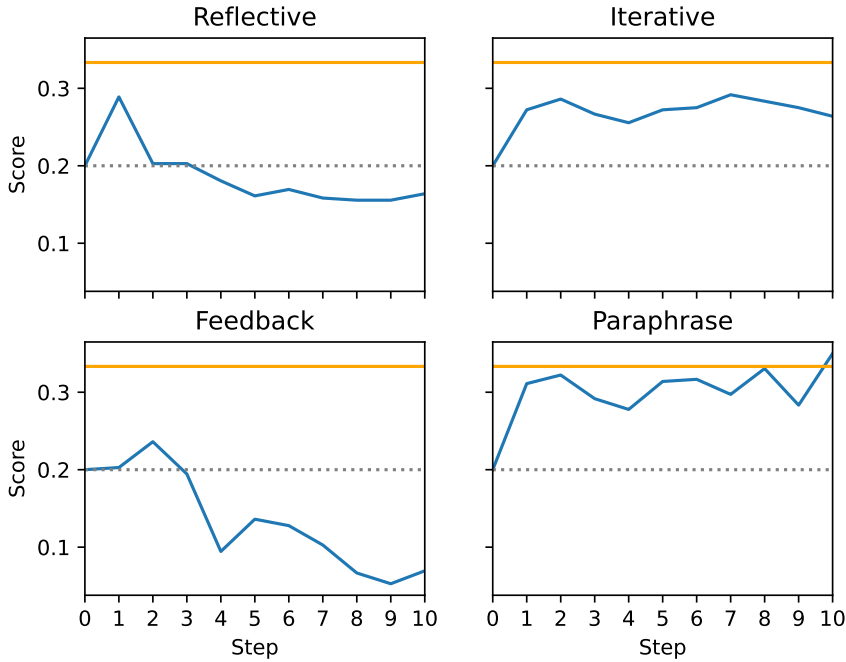


Figure 4.2: Optimization progression for the Connections dataset.

1. **Weak model:** Our optimizer model Google Gemma 3 27B is smaller than most models used in the literature, due to our limited computation budget. It could be the case that generating effective feedback and using it to improve prompts is an emergent capability of larger models.
2. **Structured generation:** We used our custom structured generation framework which allowed us to easily define *meta-prompts* and use inference techniques such as CoT, but it is possible that it made the already challenging prompt optimization task even harder and more convoluted, favoring simpler operators.
3. **Unclear comparison formulation:** In Feedback, the comparisons are made between `prompt_a` and `prompt_b`. Although we include information about whether the base prompt won the comparison, we did not state whether it was `prompt_a` or `prompt_b`. This was done under the assumption that the model could deduce this.
4. **Overfitting:** In comparisons using Reflective and Feedback, the model looks at outputs on a single task. For example, in Sequences, many prompts include instructions like “Output the last negative number”. It would probably be beneficial to compare or score attempts at multiple tasks simultaneously.

STEP	REFLECTIVE	ITERATIVE	FEEDBACK	PARAPHRASE
HB	0.70	0.70	0.70	0.70
SB	0.40	0.40	0.40	0.40
1	0.47 (0.23)	0.63 (0.36)	0.43 (0.29)	0.73 (0.38)
2	0.53 (0.39)	0.53 (0.32)	0.47 (0.32)	0.60 (0.33)
3	0.27 (0.15)	0.67 (0.43)	0.47 (0.33)	0.63 (0.37)
4	0.47 (0.21)	0.63 (0.42)	0.47 (0.32)	0.60 (0.42)
5	0.57 (0.33)	0.63 (0.40)	0.40 (0.28)	0.60 (0.38)
6	0.53 (0.35)	0.57 (0.34)	0.27 (0.19)	0.80 (0.45)
7	0.57 (0.44)	0.60 (0.31)	0.37 (0.23)	0.73 (0.37)
8	0.63 (0.46)	0.70 (0.50)	0.37 (0.25)	0.57 (0.29)
9	0.50 (0.40)	0.80 (0.37)	0.43 (0.27)	0.63 (0.44)
10	0.57 (0.41)	0.57 (0.39)	0.33 (0.21)	0.60 (0.35)

Table 4.3: Results for Sequences. Listed values are calculated using equations 4.1 and 4.2 according to the metric function in equation 3.14.

5. **Diversity and chance:** Both **Reflective** and **Feedback** use the “improve” wording as opposed to **Paraphrase** and **Iterative**, which use the “create new” wording. Although not substantiated in data, the latter operators seem to be generating more diverse prompts. This plays well with our edit distance-based pruning method.
6. **Comparisons vs. performance:** It begs whether our LLM-generated comparisons reflect the actual quality of outputs and prompts, or if they promote the model’s biases — which favor length and sounding smart [62].

By trying to establish a strong baseline, we did not utilize the best prompts for initialization. As initial prompts play a crucial role[66] in exploring the vast search space, this might have hindered our method’s performance. We hoped to achieve better initial prompt diversity by seeding the **Lamarckian** operator using **Personas** [64]. However, this diversity seems to come at the cost of diminished initial scores. We cannot answer for sure whether this trade-off is worth it.

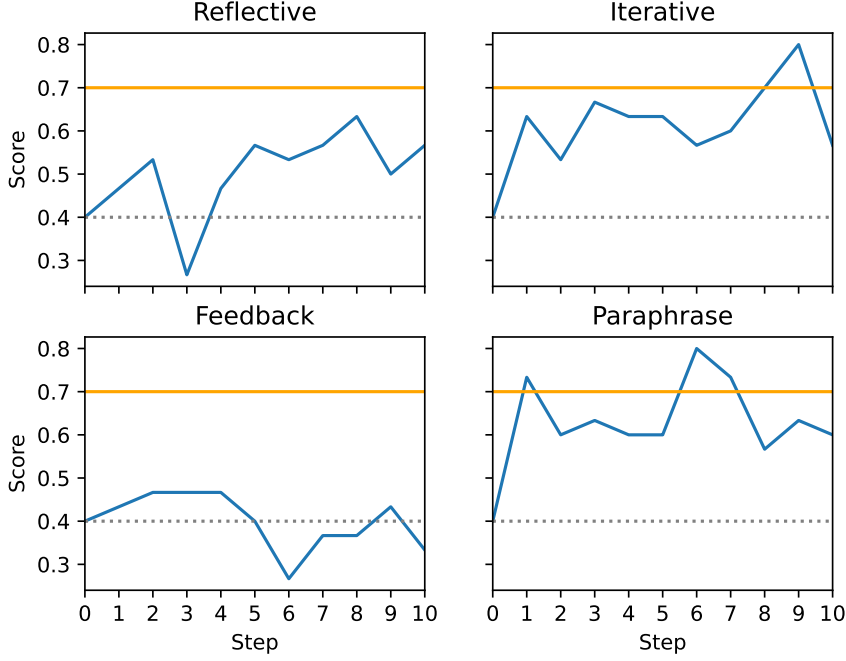


Figure 4.3: Optimization progression for the **Sequences** dataset.

Our optimizer hyperparameters were selected arbitrarily and focused on depth rather than breadth by prioritizing iteration count over population size. On all tasks except **Sequences**, the optimization process saturated quite early. This suggests that focusing on depth is misguided, and larger population sizes might be beneficial.

On **Sequences**, most of the best performing prompts include the same two few-shot examples, which have been present since initialization. Although examples are no doubt beneficial, for this particular task, they might be giving unfair advantage as some samples in $\mathcal{D}_{\text{test}}$ had the same answers, although the sequences were different.

Contrary to expectations, operators with access to the optimized prompts' outputs on $\mathcal{D}_{\text{dev}}$ did not evolve any new notable examples and did not attempt to augment existing examples. This can be explained by the fact that we do not emphasize few-shot prompting in the *Meta-prompts*.

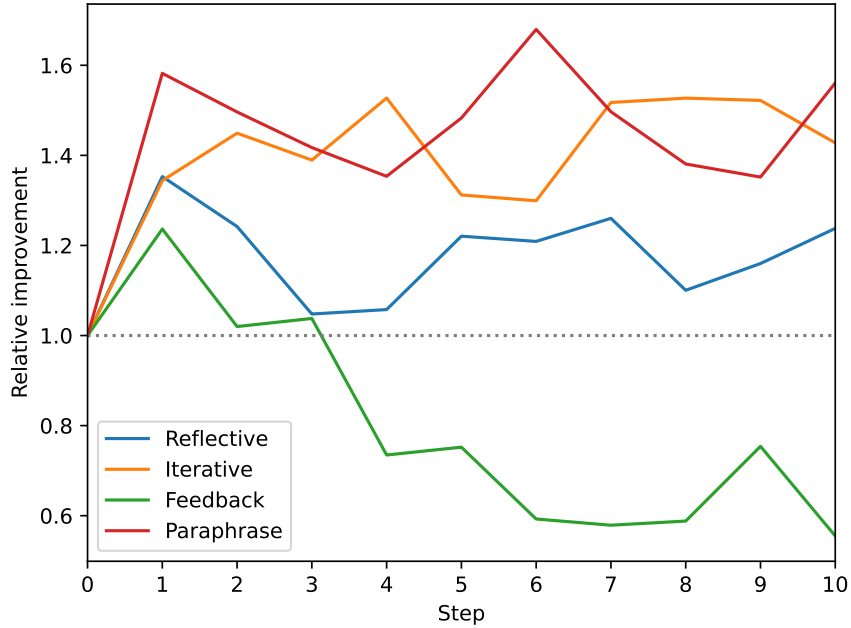


Figure 4.4: Average improvement of operators relative to the soft baseline.

4.2 Self-Supervised Optimization for Open-Ended Tasks

To showcase the applicability of our method on tasks without available gold labels, we conducted two experiments on creative applications. The only designed operator applicable in self-supervised contexts is the **Feedback** operator which performs direct output and prompt comparisons.

4.2.1 Limericks

Limerick is a short humorous poem. We designed a simple task where the LLM has to write an alliterative limerick about an animal. All words should start with the starting letter of the animal given, for example:

We provided three such examples without further context and observed the quality of the results. These examples were used to construct the initial population through the **Lamarckian** operator.

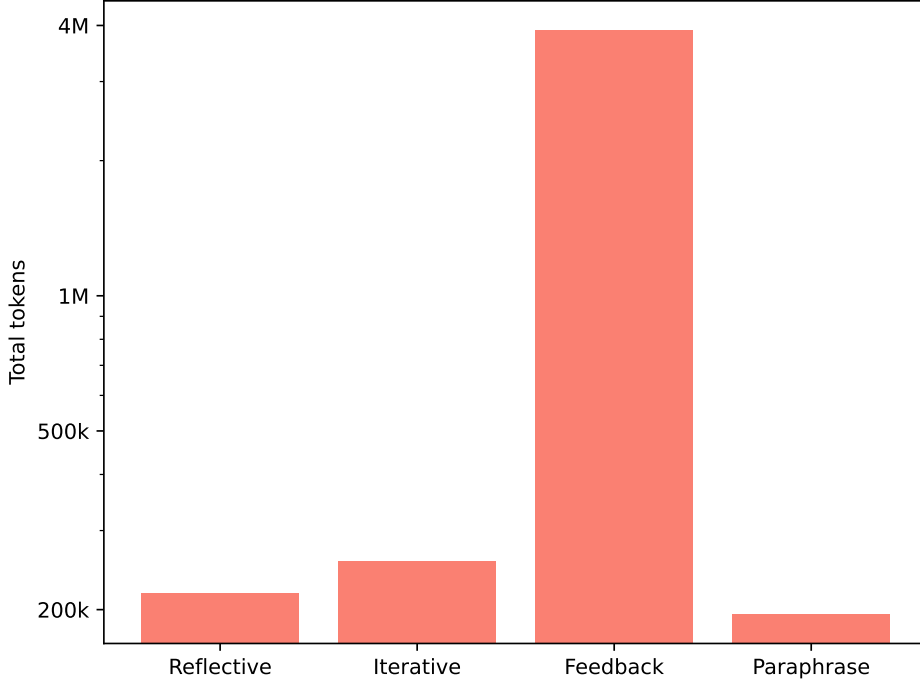


Figure 4.5: Total optimization token cost per operator. Logarithmic scale.

We ran the optimizer with population size $S = 4$, iteration count $I = 3$, batch size $B = 1$ and pruning factor $f_{\text{prune}} = 0.5$. The LLM of choice was again the Google Gemma 3 27B with $\tau = 0.75$ for $\mathcal{M}_{\text{optim}}$ and $\tau = 0.0$ for $\mathcal{M}_{\text{solve}}$.

We present the best results from the initial population (generation 0) and the last generation (3), as ranked by $\mathcal{M}_{\text{optim}}$. The initial prompts already produced satisfactory results, with the winning result making sense and having only one error in alliteration. However during the optimization, it seems that the comparison function started favoring longer results, consistent with findings of Ye et al.[62].

This problem originated in the first optimization step, where the **Feedback** operator created a prompt that instructed the LLM to “provide 3 different poem options”. Comparisons show that multiple options are preferred by the comparison function as they “showcase more variety and exploration of the theme”.

We conclude that our method failed to improve poem-generating prompts in this experiment, probably due to the ambiguous task definition, small population and batch sizes.

Input:

snake

Output:

silly snakes swiftly slide,
sneaky, slippery, side-to-side,
slurping soda,
singing sonata,
simply sparkling, super-sized!

Figure 4.6: Limerick task example

Initial:

Daring dogs delightfully dash,
dreaming deeply, a dazzling flash,
digging daily,
dancing daintily,
devoted dreamers, a delightful dash!

Step 3:

Daring dogs delightedly dash,
dancing daily, a delightful flash,
dreaming deeply,
digging deftly,
delicious dreams in a doggy splash.

Dazzling dogs dreamily doze,
delivering devotion, as everyone knows,
dancing daily,
delighting dually,
deliciously drifting where the dandelion grows.

Dynamic dogs diligently dive,
discovering delights, truly alive,
dreaming deeply,
dancing steeply,
daringly doing, to thrive and strive.

Figure 4.7: Limerick optimization result

Performance could be improved by designing a more specific operator set for creative tasks, as opposed to our general-purpose operator set.

4.2.2 Images

We employed OpenAI’s Dall-E 3[67] to demonstrate that our method can create prompts even for diffusion text-to-image models. Learning from the failure from the previous experiment, we specialized the operator set for the task of image generation. Utilizing the vision capabilities of Google Gemma 3 27B, the **Feedback** operator was modified to allow direct image comparisons.

We chose the same hyperparameters as in the Limerick experiment. For the optimizer LLM, the vision-enabled Google Gemma 3 27B was used with $\tau = 0.75$ and $\mathcal{M}_{\text{solve}}$ was a diffusion text-to-image model Dall-E 3 with image size 1024×1024 and default settings.

We selected two triples of emotionally expressive adjectives and framed the optimization as a search for a prompt that creates an image best fitting this description. The first triple is “wistful”, “hopeful”, “lonely” and the second is “graceful”, “furious”, “vulnerable”. In figures 4.8 and 4.9 the reader can find highest win-rate images from each optimization step.



Figure 4.8: Best images from each optimization step for the words “wistful”, “hopeful” and “lonely”.

All generated images reflect the target description and, as artistic taste is subjective, we will leave it to the reader to decide if the optimized prompts produced better images. We observe, that in figure 4.8, the image seems to get more personal as it zooms in on the subject. The winning images are all in a very similar style. This is in contrast to figure 4.9, where the focus seems to have shifted to photorealistic images in the first optimization step.



Figure 4.9: Best images from each optimization step for the words “graceful”, “furious” and “vulnerable”.

Note that in this case, the optimization is framed differently from previous experiments. Whereas before we optimized a prompt template which works with a whole class of tasks, in this case we are optimizing a complete prompt for a particular image description in an attempt to create a fitting image. This is a shift from compile-time to inference-time as the optimized image prompt only reflects the single image description.



Chapter 5

Conclusion

5.1 Conclusion

In this bachelor’s thesis, we conducted a comprehensive survey of cutting-edge literature, spanning inference-time scaling methods, prompt engineering and automatic prompt optimization. We conceptually framed our research topic as a part of the compile-time paradigm, contrasting against training-time and inference-time scaling.

Drawing inspiration from contemporary research trends, we developed a structured generation framework, implementing several of the discussed inference-time techniques. We established strong mathematical formalism and, building upon our structured generation framework, designed a modular prompt optimizer. Our prompt optimization framework, a population-based hill-climber algorithm, features four distinct expansion operators using *meta-prompts* - prompts which generate other prompts. Additionally, we include an initialization operator utilizing an instruction induction *meta-prompt* and a diversity-maintaining pruning feature. One of our expansion approaches is built upon a LLM-based pairwise comparison technique, suitable for self-supervised contexts.

In our primary experiment, we focused on comparing our four *meta-prompting* approaches against each other and an instruction induction baseline. We evaluate all variants of our method on three tasks from various disciplines, including a custom numerical pattern prediction task. All evaluations were repeated three times for higher statistical significance. Although the performance of some variants of our method did not meet expectations, we demonstrated that it is possible to surpass a strong instruction induction baseline even with weak initialization prompts. Our results show that LLMs’ sensitivity to input prompts is just as pronounced in the design of *meta-prompts*, highlighting the need for their careful design.

Our secondary experiment focused on optimizing prompts for creative tasks in a self-supervised context. We applied our method to creative text and image generation tasks with interesting results. These results showcase the wide applicability of prompt optimization in general.

Working on this thesis has been an excellent opportunity to gain hands-on experience with cutting-edge research in a rapidly advancing field. Although our method did not achieve impressive results, we incorporated ideas from a plethora of recent papers, and we walk away with many ideas for future research. We discuss these possible research avenues in the next section.

5.2 Future work

Prompt optimization is an exciting and exceptionally active branch of research. Our experiments inspire further work on this topic, and we will discuss possible research topics in this section.

1. **Statistical methods for evaluation:** Evaluation uses the majority of computation resources during optimization. This was most pronounced in our naive implementation of the **Feedback** operator, where we performed pairwise comparisons for every pair of prompts. In literature[51][54], researchers utilize more sophisticated evaluation methods, like Upper Confidence Bound algorithms (UCB), which can help with distribution of computational resources to the most promising prompts.
2. **Finding optimal *meta-prompts*:** The ultimate goal of this research branch is finding the general *meta-prompt*, which can create optimal prompts for any task. This is theoretically possible[47], but exceptionally difficult. Some research[20] experimented with optimizing the *meta-prompts*, but most methods hand-craft them. More rigorous experimentation and application of prompt engineering techniques, possible drawing inspiration from system prompts of foundational LLMs, might uncover much better *meta-prompts*.
3. **Improved structured generation:** Our structured generation method is in no way perfect and could represent a bottleneck, limiting the LLM’s capabilities. More work is needed to find the best format for LLM usage in this context. Although modern LLM’s are proficient JSON generators, recently leaked system prompts reveal that some top LLM’s use XML tags instead.
4. **Prompt representation structures:** More sophisticated prompt representation, like using acyclic directed graphs[54], could allow us to make more precisely defined changes to the prompts. This could help retain diversity and task relevancy as the optimized prompts grow ever longer for more complicated tasks.
5. **Applying meta-heuristics:** The majority of research made use of basic meta-heuristics, such as the hill-climber algorithms. Application of evolutionary algorithms[18] and other meta-heuristics[21] presents an exciting intersection of deep learning and classical optimization.
6. **Inference-time techniques:** For all LLM calls, we used the Chain-of-thought end-point of our inference framework. It is possible to use other inference techniques such as Tree-of-Thoughts in its place. Alternatively, dedicated reasoning models which perform Chain-of-thought automatically could be utilized.

7. **Agentic optimization:** An interesting possibility is handing the control over the optimization process over to a *manager* LLM that would control it using tools in a ReAct-like setting. We conducted preliminary experiments on this topic, but further exploration is needed.

These topics provide intriguing avenues for future research. Another possibility is increasing the experiment scale, either by making them more efficient using clever optimizations or more cost-effective models, or by increasing the computation budget.

Appendix A

Bibliography

- [1] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large language models as optimizers,” 2024.
- [2] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro, and Y. Zhang, “Sparks of artificial general intelligence: Early experiments with gpt-4,” 2023.
- [3] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling laws for neural language models,” 2020.
- [4] S. Welleck, A. Bertsch, M. Finlayson, H. Schoelkopf, A. Xie, G. Neubig, I. Kulikov, and Z. Harchaoui, “From decoding to meta-generation: Inference-time algorithms for large language models,” 2024.
- [5] OpenAI, “Openai o1 system card,” 2024.
- [6] DeepSeek-AI, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” 2025.
- [7] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncearenco, G. Sarli, I. Galynker, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle, and P. Resnik, “The prompt report: A systematic survey of prompting techniques,” 2024.
- [8] J. Zhuo, S. Zhang, X. Fang, H. Duan, D. Lin, and K. Chen, “Prosa: Assessing and understanding the prompt sensitivity of llms,” 2024.

- [9] A. Salinas and F. Morstatter, “The butterfly effect of altering prompts: How small changes and jailbreaks affect large language model performance,” 2024.
- [10] J. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, “Why johnny can’t prompt: How non-ai experts try (and fail) to design llm prompts,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI ’23, (New York, NY, USA), Association for Computing Machinery, 2023.
- [11] O. Honovich, U. Shaham, S. R. Bowman, and O. Levy, “Instruction induction: From few examples to natural language task descriptions,” 2022.
- [12] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, “Large language models are human-level prompt engineers,” 2023.
- [13] H. He, Q. Liu, L. Xu, C. Shivade, Y. Zhang, S. Srinivasan, and K. Kirchhoff, “Crispo: Multi-aspect critique-suggestion-guided automatic prompt optimization for text generation,” 2024.
- [14] T. Schnabel and J. Neville, “Symbolic prompt program search: A structure-aware approach to efficient compile-time prompt optimization,” 2024.
- [15] C. White, S. Dooley, M. Roberts, A. Pal, B. Feuer, S. Jain, R. Shwartz-Ziv, N. Jain, K. Saifullah, S. Dey, Shubh-Agrawal, S. S. Sandha, S. Naidu, C. Hegde, Y. LeCun, T. Goldstein, W. Neiswanger, and M. Goldblum, “Livebench: A challenging, contamination-limited llm benchmark,” 2025.
- [16] E. Meyerson, M. J. Nelson, H. Bradley, A. Gaier, A. Moradi, A. K. Hoover, and J. Lehman, “Language model crossover: Variation through few-shot prompting,” 2024.
- [17] S. Liu, C. Chen, X. Qu, K. Tang, and Y.-S. Ong, “Large language models as evolutionary optimizers,” 2024.
- [18] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, “Connecting large language models with evolutionary algorithms yields powerful prompt optimizers,” 2024.
- [19] W. Cui, J. Zhang, Z. Li, H. Sun, D. Lopez, K. Das, B. Malin, and S. Kumar, “Phaseevo: Towards unified in-context prompt optimization for large language models,” 2024.
- [20] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, “Promptbreeder: Self-referential self-improvement via prompt evolution,” 2023.
- [21] R. Pan, S. Xing, S. Diao, W. Sun, X. Liu, K. Shum, R. Pi, J. Zhang, and T. Zhang, “Plum: Prompt learning using metaheuristic,” 2024.

- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023.
- [23] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” 2013.
- [24] X. Wang and D. Zhou, “Chain-of-thought reasoning without prompting,” 2024.
- [25] Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H. yi Lee, and Y.-N. Chen, “Let me speak freely? a study on the impact of format restrictions on performance of large language models,” 2024.
- [26] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” 2023.
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” 2023.
- [28] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, “Dspy: Compiling declarative language model calls into self-improving pipelines,” 2023.
- [29] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *Advances in Neural Information Processing Systems* (S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, eds.), vol. 35, pp. 22199–22213, Curran Associates, Inc., 2022.
- [30] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” 2023.
- [31] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” 2023.
- [32] B. Prystawski, M. Y. Li, and N. D. Goodman, “Why think step by step? reasoning emerges from the locality of experience,” 2023.
- [33] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini, “Large language monkeys: Scaling inference compute with repeated sampling,” 2024.
- [34] Z. Zeng, Q. Cheng, Z. Yin, Y. Zhou, and X. Qiu, “Revisiting the test-time scaling of o1-like models: Do they truly possess test-time scaling capabilities?,” 2025.

- [35] R. Liu, J. Geng, A. J. Wu, I. Sucholutsky, T. Lombrozo, and T. L. Griffiths, “Mind your step (by step): Chain-of-thought can reduce performance on tasks where thinking makes humans worse,” 2024.
- [36] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” 2023.
- [37] X. Zhang, Z. Zhang, and H. Zhao, “Glape: Gold label-agnostic prompt evaluation and optimization for large language model,” 2024.
- [38] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefer, “Graph of thoughts: Solving elaborate problems with large language models,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, p. 17682–17690, Mar. 2024.
- [39] K. Misaki, Y. Inoue, Y. Imajuku, S. Kuroki, T. Nakamura, and T. Akiba, “Wider or deeper? scaling llm inference-time compute with adaptive branching tree search,” 2025.
- [40] F. Teng, Z. Yu, Q. Shi, J. Zhang, C. Wu, and Y. Luo, “Atom of thoughts for markov llm test-time scaling,” 2025.
- [41] Z. Wang, J. Zeng, O. Delalleau, D. Egert, E. Evans, H.-C. Shin, F. Soares, Y. Dong, and O. Kuchaiev, “Dedicated feedback and edit models empower inference-time scaling for open-ended general-domain tasks,” 2025.
- [42] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” 2023.
- [43] L. Reynolds and K. McDonell, “Prompt programming for large language models: Beyond the few-shot paradigm,” 2021.
- [44] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” 2020.
- [45] Y. Zhang, Y. Yuan, and A. C.-C. Yao, “Meta prompting for ai systems,” 2025.
- [46] Y. Hou, H. Dong, X. Wang, B. Li, and W. Che, “Metaprompting: Learning to learn better prompts,” 2023.
- [47] A. de Wynter, X. Wang, Q. Gu, and S.-Q. Chen, “On meta-prompting,” 2024.

- [48] K. Ramnath, K. Zhou, S. Guan, S. S. Mishra, X. Qi, Z. Shen, S. Wang, S. Woo, S. Jeoung, Y. Wang, H. Wang, H. Ding, Y. Lu, Z. Xu, Y. Zhou, B. Srinivasan, Q. Yan, Y. Chen, H. Ding, P. Xu, and L. L. Cheong, “A systematic survey of automatic prompt optimization techniques,” 2025.
- [49] Q. Ye, M. Axmed, R. Pryzant, and F. Khani, “Prompt engineering a prompt engineer,” 2024.
- [50] J. Xiang, J. Zhang, Z. Yu, F. Teng, J. Tu, X. Liang, S. Hong, C. Wu, and Y. Luo, “Self-supervised prompt optimization,” 2025.
- [51] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, “Automatic prompt optimization with "gradient descent" and beam search,” 2023.
- [52] X. Wan, R. Sun, H. Nakhost, and S. O. Arik, “Teach better or show smarter? on instructions and exemplars in automatic prompt optimization,” 2024.
- [53] E. Agarwal, J. Singh, V. Dani, R. Magazine, T. Ganu, and A. Nambi, “Promptwizard: Task-aware prompt optimization framework,” 2024.
- [54] L. Zhang, T. Ergen, L. Logeswaran, M. Lee, and D. Jurgens, “Sprig: Improving large language model performance by system prompt optimization,” 2024.
- [55] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Broman, C. Potts, M. Zaharia, and O. Khattab, “Optimizing instructions and demonstrations for multi-stage language model programs,” 2024.
- [56] J. Lehman, J. Gordon, S. Jain, K. Ndousse, C. Yeh, and K. O. Stanley, “Evolution through large models,” 2022.
- [57] M. Deng, J. Wang, C.-P. Hsieh, Y. Wang, H. Guo, T. Shu, M. Song, E. P. Xing, and Z. Hu, “Rlprompt: Optimizing discrete text prompts with reinforcement learning,” 2022.
- [58] X. Tang, X. Wang, W. X. Zhao, S. Lu, Y. Li, and J.-R. Wen, “Unleashing the potential of large language models as prompt optimizers: An analogical analysis with gradient-based model optimizers,” 2024.
- [59] D. Soylu, C. Potts, and O. Khattab, “Fine-tuning and prompt optimization: Two great steps that work better together,” 2024.
- [60] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [61] K. Paster, “Testing language models on a held-out high school national finals exam.” https://huggingface.co/datasets/keirp/hungarian_national_hs_finals_exam, 2023.

- [62] J. Ye, Y. Wang, Y. Huang, D. Chen, Q. Zhang, N. Moniz, T. Gao, W. Geyer, C. Huang, P.-Y. Chen, N. V. Chawla, and X. Zhang, “Justice or prejudice? quantifying biases in llm-as-a-judge,” 2024.
- [63] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d’Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Goyal, A. Cherepanov, J. Molloy, D. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, “Competition-level code generation with alphacode,” *arXiv preprint arXiv:2203.07814*, 2022.
- [64] T. Ge, X. Chan, X. Wang, D. Yu, H. Mi, and D. Yu, “Scaling synthetic data creation with 1,000,000,000 personas,” 2024.
- [65] G. Team, “Gemma 3 technical report,” 2025.
- [66] M. Yang, M. Li, Y. Li, Z. Chen, C. Gao, J. Zhang, Y. Li, and F. Feng, “Dual-phase accelerated prompt optimization,” 2024.
- [67] J. Betker, G. Goh, L. Jing, T. Brooks, J. Wang, L. Li, L. Ouyang, J. Zhuang, J. Lee, Y. Guo, W. Manassra, P. Dhariwal, C. Chu, Y. Jiao, and A. Ramesh, “Improving image generation with better captions,” 2023.